ISSAT Sousse

Higher Institute of Applied Science & Technology of Sousse

Republic of Tunisia

Ministry of Higher Education and Scientific Research

University of Sousse

Graduation Project Report

submitted in partial fulfilment of the requirements for the

RESEARCH MASTER'S DEGREE IN COMPUTER SCIENCE

Speciality: **Intelligent Pervasive Systems**

Evaluating Coding Agent Quality: Application to Software Design Principles

---

Prepared by

Akram Ben Fekih

Supervised by

Dr. Maha Khemaja

Jury

President: _____

Presented on _____

Academic Year 2025 / 2026

Topic code / ID: _____

To my family, for their unwavering support, unconditional love, and constant encouragement. You are my source of inspiration.

To my teachers, for their shared knowledge, patience, and commitment to my learning. You have guided me to success.

To my supervisors, for their expertise.

To all those who have crossed my path and contributed, directly or indirectly, to the realization of this project, I extend my sincere thanks to you.

Finally, to all those who have been part of this academic journey with me, I dedicate this work to you. Your contributions have been invaluable.

Acknowledgements

First and foremost, I would like to express my gratitude to the Almighty for granting me the courage and patience needed to complete this work.

I thank my academic supervisor, Dr. Maha Khemaja, for her guidance and her patience. Her criticism shaped how this work is structured and improved it in every chapter.

I thank the members of the jury for the time they have given to reading and evaluating this work.

I thank my professors, who spent their effort on teaching me, and without whom I could not have finished this.

Finally, I thank everyone who helped, directly or indirectly, to bring this work about.

ملخص

باتت الشفرة الصادرة عن عملاء البرمجة الذاتيين تُقدَّم إلى مشاريع حقيقية، غير أنّ معايير التقييم الحالية تكاد تقتصر على الصحة الوظيفية، تاركةً جودة العمارة البرمجية دون توصيف. تقيس هذه الأطروحة مدى التزام الشفرة الصادرة عن Claude Code وGitHub Copilot وOpenAI Codex بمبادئ SOLID الخمسة، اعتمادًا على 4245 طلب دمج بلغة جافا و4946 طلب دمج بلغة بايثون، تشمل المدمجة والمرفوضة والمفتوحة.

واعتُمد لكل مبدأ مؤشرٌ بنيويٌّ حتميٌّ جرى قياس صدقه لا افتراضه: إذ خضعت 2475 حالة لحكم قراءةٍ مستقلٍّ يسجّل سبب كل حكم. وتراوحت الدقّة بين 0.330 و0.892 في جافا، وبين 0.110 و1.000 في بايثون.

وفي اللغتين يطغى حجم التغيير على كل المبادئ، والفرق الوحيد بين العملاء الذي ظهر في اللغتين هو ارتفاع معدّل Claude Code في مبدأ المفتوح/المغلق. ومعظم المخالفات كانت موجودة قبل تغييرات العملاء: ففي الشفرة المعدَّلة 183 حالة رفضٍ للوراثة في جافا مقابل 16 طلب دمج أدخل حالة جديدة، و176 مقابل خمسة في بايثون. ولا تكاد شفرة جافا المدمجة تختلف عن غير المراجَعة، أمّا في بايثون فالشفرة المدمجة أنظف.

الكلمات المفتاحية: عملاء البرمجة، مبادئ SOLID، العمارة البرمجية، التحليل الساكن، جافا وبايثون.

Abstract

Autonomous coding agents now author code submitted to real software projects, yet existing benchmarks measure functional correctness almost exclusively. This thesis measures whether code from Claude Code, GitHub Copilot and OpenAI Codex follows the five solid principles, across **4,245 Java pull requests** and **4,946 Python pull requests**, merged, rejected and open.

Each principle is a deterministic structural proxy whose construct validity is measured: **2,475 entities** were read and labelled by an independent judge that recorded a reason for every verdict. Precision ranges from 0.330 to 0.892 in Java and 0.110 to 1.000 in Python.

In both languages change size dominates every principle, roughly doubling the odds of a flag per doubling of files touched, and the only agent difference found in both is Claude Code's higher rate on Open/Closed. Most violations predate the agents' changes: the touched code holds 183 refused-bequest violations in Java against 16 pull requests that introduced one, and 176 against five in Python. Merged and unreviewed Java code look alike; merged Python code is cleaner.

Keywords: coding agents, SOLID principles, software architecture, static analysis, empirical software engineering, Java and Python.

Résumé

Les agents de codage autonomes produisent du code soumis à de vrais projets logiciels, mais les référentiels existants mesurent presque exclusivement la correction fonctionnelle. Cette thèse mesure si le code de Claude Code, GitHub Copilot et OpenAI Codex respecte les cinq principes solid, sur **4 245 *pull requests* Java** et **4 946 *pull requests* Python**, fusionnées, rejetées et ouvertes.

Chaque principe est un indicateur structurel déterministe dont la validité de construit est mesurée : **2 475 entités** ont été lues et annotées par un juge indépendant. La précision varie de 0,330 à 0,892 en Java et de 0,110 à 1,000 en Python.

Dans les deux langages la taille du changement domine chaque principe, et le seul écart entre agents retrouvé dans les deux est le taux plus élevé de Claude Code sur Ouvert/Fermé. La plupart des violations précèdent les changements des agents : le code touché contient 183 refus d'héritage en Java contre 16 *pull requests* qui en introduisent un, et 176 contre cinq en Python. Le code Java fusionné ressemble au code non revu ; le code Python fusionné est plus propre.

Mots-clés : agents de codage, principes SOLID, architecture logicielle, analyse statique, génie logiciel empirique, Java et Python.

Contents

1 Introduction **1**

1.1 Research Context 2
1.2 Problem Statement 3
1.3 Research Questions 4
1.4 Research Objectives 4

2 Literature Review **6**

2.1 Foundations of Software Quality and Design 7
2.1.1 Design Principles 7
2.1.2 Code Smells and Their Detection 8
2.1.3 Metrics 8
2.2 Evolution of Coding Agents 9
2.2.1 First-Generation Assistants (2021–2022) 9
2.2.2 Agentic Tools (2023–2025) 9
2.2.3 Functional Benchmark Performance 10
2.3 Evaluation of Design Quality in AI-Generated Code 10
2.3.1 Principle Adherence 10
2.3.2 Code Smells in Agent Output 10
2.3.3 Repository-Scale and Acceptance Studies 11
2.3.4 Prompting and Mitigation 12
2.4 Comparative Synthesis 12
2.5 Critical Analysis and Motivation for This Work 15

3 Research Methodology **16**

3.1 Research Design Overview 17
3.1.1 Research Paradigm 18
3.1.2 Study Type 18
3.2 Research Questions 18
3.3 Research Strategy 21
3.3.1 A Benchmark over Real Pull Requests 21
3.3.2 One Deterministic Detector per Principle 21
3.3.3 The Before/After Delta 21
3.3.4 Validation by a Reading Judge 22

3.3.5 Screens, Suppressors and Refinements . . . 22
3.3.6 Controlling for Repository Mix . . . 23
3.3.7 Language Selection . . . 23
3.4 Mapping SOLID to Structural Detectors . . . 24
3.5 Validity . . . 25
3.6 Ethical Considerations . . . 27

4 Experimental Setup and Results 28
4.1 Overview . . . 29
4.2 Dataset . . . 29
4.2.1 Source . . . 29
4.2.2 Curation . . . 29
4.2.3 Agent Selection . . . 31
4.2.4 Reconstruction and Attrition . . . 31
4.2.5 Resolution Quality . . . 32
4.3 Toolchain . . . 32
4.3.1 Selection . . . 32
4.3.2 Tools Used . . . 33
4.3.3 Experimental Environment . . . 33
4.3.4 Research and Writing Tools . . . 35
4.4 Detectors, Thresholds and Equations . . . 35
4.4.1 SRP: God Class Rubric with a Cohesion Suppressor . . . 36
4.4.2 OCP: Dispatch Chain Length with a Closed-Set Suppressor . . . 38
4.4.3 LSP: Override Compatibility over the Resolved Hierarchy . . . 39
4.4.4 ISP: Contract Width and Implementer Stub Ratio . . . 40
4.4.5 DIP: Import Graph Delta . . . 40
4.4.6 Component Zones: a Package-Level Description . . . 41
4.5 Analysis Pipeline . . . 42
4.6 Validation Procedure . . . 43
4.6.1 A Second Check: Independent Analysers . . . 43
4.7 Results . . . 44
4.7.1 Detector Validation (RQ2) . . . 44
4.7.2 Agreement with Independent Analysers . . . 47
4.7.3 Agent Differentiation (RQ1) . . . 49
4.7.4 Locus of Risk (RQ1) . . . 54
4.7.5 Cross-Cutting Findings . . . 55
4.8 Results in Python (RQ1 and RQ2) . . . 57
4.8.1 Corpus . . . 57
4.8.2 Changes to the Instrument . . . 58

4.8.3 Detector Validation . . . 59
4.8.4 Agent Differences under the Joint Model . . . 61
4.8.5 Merged and Rejected Code . . . 62
4.8.6 Locus of Risk . . . 63
4.8.7 The Java Refinements Applied to Python . . . 64
4.8.8 Summary of the Two Languages . . . 64

5 Conclusion **66**
5.1 Summary of the Work . . . 67
5.2 Answers to the Research Questions . . . 67
5.3 This Study Against the State of the Art . . . 70
5.4 Contributions . . . 72
5.5 Practical Implications . . . 73
5.6 Limitations . . . 73
5.7 Future Work . . . 76
5.8 Closing Remark . . . 77

Bibliography **78**

A Pipeline Run Order and Reproducibility **82**

List of Figures

1.1 AI coding tools from 2021 to 2025. 2

2.1 The five SOLID principles. 7

3.1 Intra-unit and inter-unit violations. 20
3.2 The before/after delta for Dependency Inversion. 22

4.1 Martin's abstractness and instability plane. 41
4.2 The measurement pipeline. 42
4.3 Detector precision and weighted recall in Java and Python. 45
4.4 SRP flag rate for authored and edited classes. 53
4.5 Violations present and introduced, in Java and Python. 55

List of Tables

2.1 Closely related studies. 13

3.1 Study type. 19
3.2 Research questions and how each is answered. 20
3.3 From principle to detector (details in Chapter 4). 26

4.1 Java corpus by agent and outcome. 30
4.2 How the Java pull requests that were not merged ended. 31
4.3 From the curated Java corpus to the analysable corpus. 31
4.4 Analysis toolchain. 34
4.5 Machine and software environment. 35
4.6 Research and writing tools. 36
4.7 What each principle asks and what the Java detector computes. 37
4.8 Java detector agreement with the reading judge. 44
4.9 Java detector precision by agent. 46
4.10 ISP precision by contract width. 47
4.11 The two refinements on the derivation sample and on a holdout. 47
4.12 Rank correlation with CK. 48
4.13 Agreement between three God Class detectors. 48
4.14 Java flag rate by agent. 50
4.15 Java flag rate by agent and change size. 50
4.16 Adjusted odds ratios from the joint model for Java. 51
4.17 Java SRP flag rate for authored and edited classes. 52
4.18 Java flag rate by outcome stratum. 53
4.19 Java violations present and introduced. 54
4.20 OCP precision by dispatch chain length. 56
4.21 Analysable Python pull requests by agent and outcome. 57
4.22 From the curated Python corpus to the analysable corpus. 58
4.23 Detector thresholds in Java and Python. 59
4.24 Python detector agreement with the reading judge. 60
4.25 Python precision by agent. 61
4.26 Python flag rate by agent. 61
4.27 Adjusted odds ratios from the joint model for Python. 62

4.28 Python flag rate by outcome stratum. 62
4.29 Python violations present and introduced. 63
4.30 The two Java refinements applied to the Python labels. 64
4.31 Java results tested in Python. 65

5.1 Earlier findings compared with this study. 71

List of Acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CBO	Coupling Between Objects
CK	Chidamber–Kemerer (metrics suite)
DIP	Dependency Inversion Principle
DIT	Depth of Inheritance Tree
GRASP	General Responsibility Assignment Software Patterns
ISP	Interface Segregation Principle
JDBC	Java Database Connectivity
LCOM	Lack of Cohesion in Methods
LLM	Large Language Model
LSP	Liskov Substitution Principle
NOC	Number of Children
OCP	Open/Closed Principle
OR	Odds Ratio
P / R	Precision / Recall
PMD	Static-analysis tool for Java source code
PR	Pull Request
RFC	Response For a Class
RQ	Research Question
SOLID	Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
SRP	Single Responsibility Principle
TF-IDF	Term Frequency–Inverse Document Frequency
TP / FP / FN	True Positive / False Positive / False Negative
WMC	Weighted Methods per Class

Chapter

1

Introduction

1.1 Research Context

AI tools for software development have changed in what they do. GitHub Copilot (2021) and Amazon CodeWhisperer (2022) completed the next line or block, and the developer decided what to keep. Newer tools work at the level of a whole task. Claude Code (Anthropic, 2025), Cursor's agent mode, Devin (Cognition, 2024), OpenAI Codex CLI (2025), Google Jules (2025) and open-source tools such as OpenCode and Aider read an issue, plan steps, edit files, run tests and revise their changes without a human approving each step. This loop, in which a language model alternates reasoning with tool calls, was described as the ReAct pattern (Yao et al., 2023). As Figure 1.1 shows, assistants (above the axis) suggest code in the editor, while agents (below the axis) take a task and return a finished change. The first half of 2025 is enlarged because the three agents studied in this thesis (in gold) were released in that period.

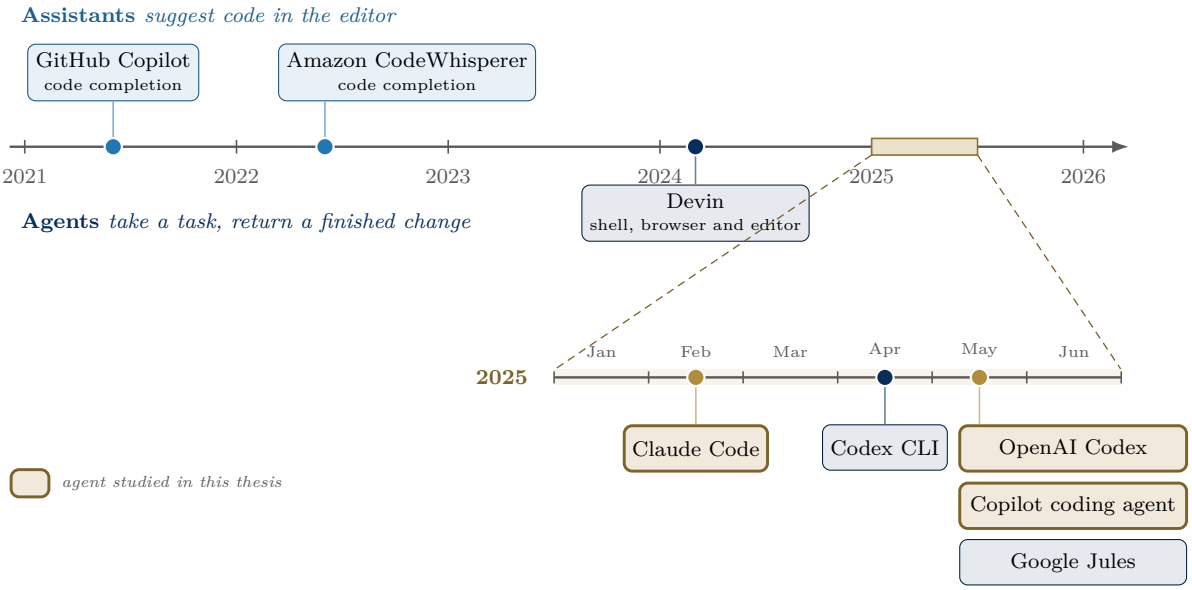


Figure 1.1: AI coding tools from 2021 to 2025.

A developer can now hand an agent a feature description and receive a pull request. The informal term *vibe coding* describes this way of working: the developer states the intent and the agent writes the implementation. Konrad et al. (2026) describe a related effect, *vibe architecting*, in which structural decisions follow from how a prompt is worded and leave no recorded rationale.

These agents already contribute to real projects. The AIDev dataset (Li et al., 2025) collects pull requests opened by autonomous coding agents on public GitHub repositories, and studies built on it work with thousands of such requests. Because agent-written code is reviewed and merged into real software, its structural quality can be studied on real data. This thesis uses the same dataset.

Uptake and merge figures describe use, which is only one part of software quality.

Software engineering has a longer vocabulary for design: Brooks's analysis of accidental complexity (Brooks, 1975), Martin's design principles (Martin, 2000), later grouped under the acronym SOLID, and the code smells catalogued by Fowler et al. (1999). Code can pass its tests and still be tightly coupled and expensive to change, and most of the cost of software is spent on maintenance after the first release.

Recent studies suggest that AI assistance affects this side of quality. Xu et al. (2025) report a shift in how core developers split their time between writing and reviewing code after Copilot was introduced, and Zhu et al. (2026b) describe a *Volume-Quality Inverse Law*: in AI-generated repositories, code volume predicts structural degradation, and neither functional correctness nor detailed prompting prevents it.

1.2 Problem Statement

Coding agents are mostly evaluated on properties that are easy to test automatically, and less on the properties that decide how expensive code will be to maintain.

Benchmarks measure function, not design. HumanEval (Chen et al., 2021), MBPP, BigCodeBench (Zhuo et al., 2025) and SWE-bench (Jimenez et al., 2024) all ask whether generated code passes a set of tests, usually for one function or one issue. A pass rate is well defined and reproducible, but it says nothing about whether the surrounding code is well structured or whether a maintainer would accept the change. The distance between the two has been measured: Whitfill et al. (2026) found that roughly half of the agent patches that pass SWE-bench Verified would not be merged by maintainers.

Structural quality is rarely compared across agents. Studies that do look at the structure of AI-generated code ask language models to detect SOLID violations (Pehlivan et al., 2025), analyse large projects generated by one tool (Kashif et al., 2026), or describe recurring defects in agent-written code (Zhu et al., 2026b). None of them compares several agents on the same corpus of real contributions.

The agent matters, not only the model. Most evaluations vary the language model and keep the agent software fixed. Ben Sghaier et al. (2026) show that the agent software has an effect of its own, so someone choosing a tool chooses an agent, not only a model. This thesis compares three agents: Claude Code, GitHub Copilot and OpenAI Codex.

In short, current evaluation practice says little about the design quality of the code agents contribute to real repositories. Adoption decisions may therefore rest on functional scores alone, while maintenance costs accumulate elsewhere. The problem addressed by this thesis is whether the code that coding agents contribute to real projects follows

established object-oriented design principles, in addition to working, and whether the agents differ in this respect.

1.3 Research Questions

The study answers two research questions. Chapter 3 (Section 3.2) describes how each is answered.

RQ1 (Adherence and differentiation)

Do coding agents differ in how well their pull requests adhere to the five SOLID principles, and how many of the design problems in the code they touch do they introduce themselves, in Java and in Python?

RQ2 (Measurability)

Can SOLID adherence be measured at repository scale with deterministic structural detectors, and how well do these detectors agree with an independent reading judge and with established analysers?

1.4 Research Objectives

The main objective is to find out whether code written by coding agents follows established object-oriented design principles, in addition to working. The thesis does so in two languages, Java and Python.

One scoping decision bounds every later claim. The five SOLID principles describe classes and the relations between them, so all five measurements work below the level of the package. When the thesis uses the word architecture, it means design structure at this level, not the decomposition of a system into components and layers.

The objectives are:

O1. Build a reproducible framework that measures adherence to the five SOLID principles (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion) on real pull requests written by coding agents, using one deterministic structural detector per principle, in Java and in Python.

O2. Build the corpus from all three outcomes a pull request can have: merged, rejected and still open. A corpus of merged requests shows what maintainers accepted, which is not the same as what agents produced.

O3. Measure how well each detector agrees with an independent reading of the code. A judge reads the source of a stratified sample of flagged and unflagged entities, and precision and recall are reported for each principle. The judge is a large

language model, a technique validated for SOLID detection by Pehlivan et al. (2025), so every figure is agreement between the detector and the judge.

O4. Compare three agents, Claude Code, GitHub Copilot and OpenAI Codex, with rates reported both pooled and averaged over repositories.

O5. Determine where design problems are located: inside one unit (a class or a method) that a single change can write in full, or in a relation between units (a dependency, an interface, an override) that a single change rarely writes on both sides.

O6. Carry out the whole study in two languages, Java and Python, with the same agents, curation rules, validation procedure and statistical model, to find out which results hold in both.

Three design choices frame the study. First, generating many new repositories with each agent would require compute and API budgets that were not available. The study therefore measures code the agents actually submitted to open-source projects, taken from the AIDev dataset (Li et al., 2025). This is cheaper, it reflects real tasks, and each pull request is a bounded change made by one agent.

Second, the study is carried out in two languages, and they play different roles. A common objection to measuring SOLID is that the principles fit poorly in languages that do not require object orientation. Java answers that objection: classes are compulsory, `interface` is a keyword, `@Override` is explicit and subtyping is nominal, so each principle has a direct counterpart in the syntax. Python is the opposite case: classes are optional, contracts are conventions, and no compiler checks an override against its base. A result found in both languages is unlikely to be a side effect of Java's type system. Only the fact-extraction layer of the framework depends on the language, so the Python study uses its own extractor and recalibrated thresholds and changes nothing else.

Third, the agent set follows from the data. The same three agents are used in both languages so that every cross-language comparison has the same agents on both sides. Section 4.2.3 explains why the other agents were left out.

Chapter

2

Literature Review

2.1 Foundations of Software Quality and Design

Evaluating coding agents against design criteria requires a clear account of those criteria. This section covers design principles, code smells and the metrics used to detect them.

2.1.1 Design Principles

Robert C. Martin's design principles (Martin, 2000, 2003), grouped under the acronym SOLID, are the most widely taught framework for object-oriented design. As Figure 2.1 shows, each principle can be pictured through the typical shape of its violation (in the interface card, dotted members are stubs). The Single Responsibility Principle (SRP) states that a class should have one reason to change. The Open/Closed Principle (OCP) states that a module should be open for extension and closed for modification. The Liskov Substitution Principle (LSP) requires that a subtype can be used wherever its base type is expected. The Interface Segregation Principle (ISP) states that clients should not depend on methods they do not use. The Dependency Inversion Principle (DIP) states that high-level modules should depend on abstractions, not on concrete details. These are heuristics, but they connect to a long line of metrics work: cohesion, coupling and inheritance depth can all be measured, which is what makes automated assessment possible. Larman's GRASP patterns (Larman, 2004) give finer guidance on assigning responsibilities, such as Information Expert, Low Coupling and High Cohesion.

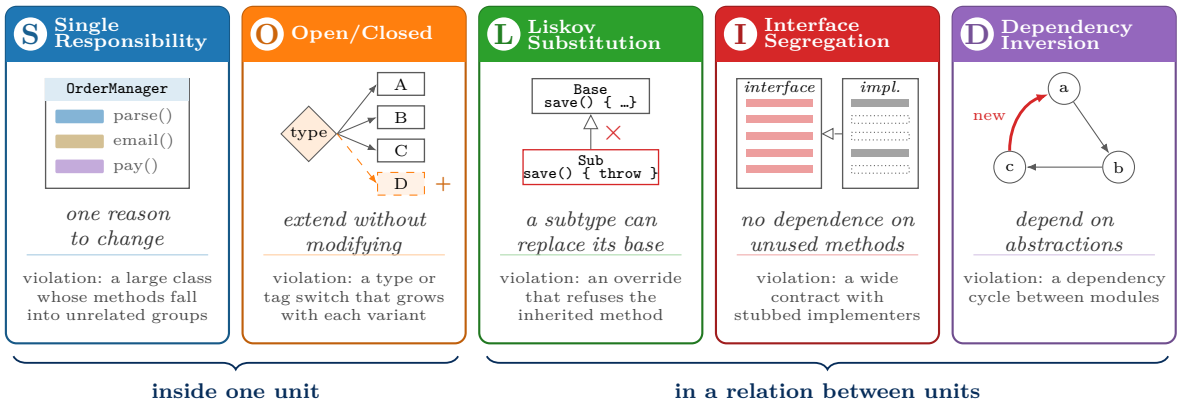


Figure 2.1: The five SOLID principles.

Evidence on how well agents follow these principles is recent. The NITR benchmark (Zhu et al., 2026a) tests whether agent edits keep code maintainable, over 21 curated C++ repository probes spanning nine maintainability dimensions, and finds dependency control (4.3% success) harder than responsibility decomposition (15.2%). Pehlivan et al. (2025) prompt four language models to detect SOLID violations in 240 manually validated examples: Single Responsibility is the easiest principle to detect, Dependency Inversion the hardest, and accuracy drops as code becomes more complex.

Much of the agent-evaluation work reviewed in this chapter, including most of the 2026 studies in Table 2.1, is available only as preprints. These studies are cited because they are the closest related work. Where the thesis relies on one, it names the specific result it uses.

2.1.2 Code Smells and Their Detection

Fowler et al. (1999) popularised the term *code smell* for patterns such as Long Method, God Class, Data Clumps and Feature Envy, which point to design problems without being bugs. Suryanarayana et al. (2014) later catalogued design smells by the design principle they violate; for example, *Missing Hierarchy* is conditional logic on type codes used where a class hierarchy could hold the variation. DesigniteJava is a tool that detects smells from that catalogue.

The link between smells and maintenance cost has been studied but is not uniform. Khomh et al. (2012) find that classes participating in antipatterns are more change-prone and fault-prone than other classes. Olbrich et al. (2010) find that God Classes and Brain Classes change more often and contain more defects, but that after normalising for size they change less often and contain fewer defects than other classes. Size alone is therefore a weak indicator of a design problem, and a detector that relies on it should be read with caution.

Automated detection usually works through metric thresholds. Marinescu (2004) formalised this as *detection strategies*: rules that combine several metrics, each compared with a threshold, to locate a specific design flaw such as a God Class. Tools such as PMD, SonarQube, CodeScene and the CK metrics suite apply rules of this kind to many codebases at once.

Zhu et al. (2026b) study smells in AI-generated code. They report a *Volume-Quality Inverse Law*, in which code volume is a near-perfect predictor of structural degradation, and a *Reasoning-Complexity Trade-off*, in which more capable models generate more bloated and more coupled code. They conclude that addressing this requires explicit architectural foresight during generation.

2.1.3 Metrics

The Chidamber and Kemerer (CK) suite (Chidamber and Kemerer, 1994) defines Weighted Methods per Class (WMC), Depth of Inheritance Tree (DIT), Number of Children (NOC), Coupling Between Objects (CBO), Response For a Class (RFC) and Lack of Cohesion in Methods (LCOM). LCOM has several later variants, including the connected-components form LCOM4 (Hitz and Montazeri, 1995) and the normalised LCOM* (Henderson-Sellers, 1996). For packages, Martin's instability and abstractness metrics (Martin, 2003) measure how much a package depends on others and how abstract it is.

RealBench (Li et al., 2026) measures architectural quality in repository-level generation. Its 61 repositories come with natural-language requirements and UML designs, and generated code is scored on an Architecture@k metric from 0 to 4. The best score across the models studied is 2.29. Test pass rates fall with repository size, from 25.48% for repositories of up to 500 lines to 5.81% for repositories of 2,000 lines or more.

2.2 Evolution of Coding Agents

AI coding tools have changed quickly (Figure 1.1, p. 2). The newer tools take on different kinds of task, and this affects how they should be evaluated.

2.2.1 First-Generation Assistants (2021--2022)

GitHub Copilot, released in 2021 and built on OpenAI's Codex model, suggested the next line or function from the surrounding code. Amazon CodeWhisperer (2022) added security scanning and reference tracking for suggestions that resemble licensed code. Both reacted to what the developer was typing. HumanEval (Chen et al., 2021), which measures pass@k on 164 hand-written Python problems, suited this kind of tool.

2.2.2 Agentic Tools (2023--2025)

Devin (Cognition, 2024) was presented as an agent that completes tasks end to end, using a shell, a browser and an editor. Cursor added repository indexing and agentic editing to an IDE. Claude Code (Anthropic, 2025) runs as an agent in the terminal. OpenCode (2025) is an open-source agent, and OpenAI's Codex CLI (2025) runs a local agent loop with shell access.

These tools keep state across a session, break a task into steps, call tools and retry after failures. A function-level benchmark such as HumanEval does not exercise these abilities.

Few peer-reviewed comparisons of these tools exist. Practitioner reports describe design choices inside them. Anthropic's report on context engineering (Anthropic, 2025) describes a language model's context as a limited attention budget and names bloated tool sets, which leave the model unsure which tool to use, as one of the most common failure modes it observes. The report comes from the vendor of one of the agents measured in this thesis, so it is cited only for this design argument and not as evidence. Ben Sghaier et al. (2026) provide empirical support: with the model held fixed, quality changes across releases of the agent software.

2.2.3 Functional Benchmark Performance

On function-level benchmarks agents score highly. AgentCoder (Huang et al., 2023), which uses separate programmer, test designer and test executor agents, reaches 96.3% pass@1 on HumanEval with GPT-4. HumanEval tests single functions, where design problems are least visible.

Harder benchmarks give lower scores. BigCodeBench (Zhuo et al., 2025) contains 1,140 tasks that require calls to 139 libraries. Across about 60 models the best score is 60%, against 97% for humans. LiveCodeBench (Jain et al., 2024) uses problems published after a model's training cut-off in order to detect contamination. SWE-EVO (Le et al., 2025) contains 48 long-horizon evolution tasks that change 21 files on average. GPT-5.4 with OpenHands resolves 25% of them, while GPT-5.2 resolves 72.80% of SWE-bench Verified.

2.3 Evaluation of Design Quality in AI-Generated Code

A growing body of work examines structural properties of agent output. This section groups it by what it measures.

2.3.1 Principle Adherence

Pehlivan et al. (2025) evaluate language models as detectors of SOLID violations (see Section 2.1.1). Kashif et al. (2026) generate large projects with Cursor, averaging 16,965 lines of code and 114 files. The projects reach 91% functional correctness on average, yet CodeScene reports 1,305 design issues and SonarQube 3,193, most often code duplication, high complexity and large methods, which the authors relate to SRP and to separation of concerns.

Konrad et al. (2026) describe how agents make architectural decisions. Agents such as Claude Code and Codex choose frameworks, configure databases and set up authentication, and the authors treat each of these choices as an architectural decision. Without explicit rules, agents fall back on patterns learned from training data, and the decisions leave no design documents behind. This suggests that agent-written code can end up with an unplanned structure.

2.3.2 Code Smells in Agent Output

Zhu et al. (2026b) argue that AI-generated code carries a distinct "machine signature" of defects, including broken abstractions (an interface created and then bypassed through concrete types), systematic duplication, over-engineering and verbosity. Broken abstractions relate to the Dependency Inversion and Open/Closed principles.

Some design problems have a structural form and can be detected in principle. Others concern intent: code with an unremarkable shape that was not written to any coherent design. A static rule cannot detect the second kind, so a detector based on structure has to be checked against a reader of the code before its flags are treated as verdicts on design.

2.3.3 Repository-Scale and Acceptance Studies

ProjDevBench (Lu et al., 2026) evaluates six coding agents on 20 project-development problems in eight categories, combining online-judge testing with LLM-assisted code review. The overall acceptance rate is 27.38%. Agents handle basic functionality and data structures but struggle with complex system design, time complexity and resource management. With 20 problems, a comparison between individual agents is limited.

Two studies use the AIDev corpus that this thesis also uses. Mazloomzadeh et al. (2026) compare agentic and human pull requests over successive quarters: how their merge rates differ, which tasks agents are given and how the characteristics of the requests change. Cynthia et al. (2026) analyse 1,210 merged agent bug-fix pull requests from Python repositories with SonarQube, comparing the base and merged commits so that only issues introduced by the pull request are counted. Three of their results bear on this thesis. Differences between agents in raw issue counts largely disappear after normalising by code churn. Code smells are more common than bugs, especially at critical and major severity. And merge success does not reliably reflect post-merge code quality.

Sawada et al. (2026) study what happens to agent-generated code after it is merged, over more than 1,000 files and about 3,200 changes in 100 popular repositories from AIDev. AI-generated files are maintained less often than human-written files, and each update touches a small fraction of the file. Updates to AI-written files are mostly feature extensions, updates to human-written files mostly bug fixes, and human developers carry out most of the maintenance in both cases. Agent code is therefore extended by people other than its author, which is the situation in which its structure matters.

SWE-bench (Jimenez et al., 2024) measures how many GitHub issues a system resolves with the associated tests passing. It evaluates repair of single issues and does not evaluate design. Whitfill et al. (2026) show how far test-based scores are from maintainer judgement: roughly half of the SWE-bench Verified patches that pass the tests would not be merged, and maintainer decisions are about 24 percentage points below the automated score. Because the agents could not respond to review feedback, the authors do not present the gap as a fixed capability limit.

SlopCodeBench (Orlanski et al., 2026) follows 15 agents through 36 problems and 196 checkpoints in which each task extends the agent's own earlier solution. No agent solves a problem end to end, and the best agent passes 14.8% of checkpoints. Structural erosion increases in 77% of trajectories and verbosity in 75.5%, and agent code is 2.3 times more

verbose than comparable open-source Python code.

2.3.4 Prompting and Mitigation

Prompt design changes detection quality: in Pehlivan et al. (2025) an ensemble prompt works best for OCP and an example-based prompt for DIP. For generation, SlopCodeBench reports that quality-aware prompts reduce initial verbosity and erosion by up to a third but do not change the rate at which quality degrades (Orlanski et al., 2026).

Linting loops, in which an agent runs a checker and revises its code before committing, are recommended in practitioner writing. We found no controlled study of their effect on design quality, as opposed to lint compliance. Such a loop can only enforce what its checker can express, and a checker based on structure cannot express design intent.

2.4 Comparative Synthesis

As Table 2.1 shows, the most closely related studies differ in what they evaluate and measure, and each stops short of assessing design quality across several agents on real contributions. The last column of the table states where each study's scope stops.

Table 2.1: Closely related studies.

Study	Systems evaluated	Measures	Main findings	Limitation
Pehlivan et al. (2025)	4 LLMs	LLM detection of SOLID violations (240 examples)	SRP easiest to detect, DIP hardest; accuracy falls with complexity	Detection on examples, not measurement of agent contributions
Kashif et al. (2026)	Cursor	Design issues in generated projects	91% functional correctness, yet 1,305 CodeScene and 3,193 SonarQube issues	One tool; no comparison between agents
Konrad et al. (2026)	Several agents	Implicit architectural decisions	"Vibe architecting"; one task gave 141 to 827 lines over 2 to 6 files	Qualitative; no measurement over a corpus
Lu et al. (2026)	6 coding agents	Online-judge tests and LLM code review	27.38% acceptance; weak on system design	20 problems; no per-principle measurement
Jimenez et al. (2024)	Language models and agents	Issues resolved with tests passing	Standard issue-level benchmark	Issue repair only; no design measurement
Whitfill et al. (2026)	Agents on SWE-bench Verified	Maintainer merge decisions	About half of passing patches would not be merged; 24-point gap	Merge verdict only
Zhu et al. (2026a)	Agents	Maintainability tasks with a structural oracle, 21 C++ probes	4.3% dependency control, 15.2% responsibility decomposition; 13.3% pass tests but fail the oracle	Benchmark tasks in one language, not contributions to real projects

(Table 2.1 continued)

Study	Systems evaluated	Measures	Main findings	Limitation
Orlanski et al. (2026)	15 agents	Verbosity and erosion across checkpoints	Best agent passes 14.8% of checkpoints; erosion rises in 77% of trajectories	Iterative benchmark tasks; not SOLID-specific
Zhu et al. (2026b)	LLM- and agent-driven projects	Code and architecture smells	Machine signature; Volume-Quality Inverse Law	Smells, not principles
Cynthia et al. (2026)	Agent bug-fix PRs in AIDev	SonarQube issues introduced by merged PRs	Agent differences vanish after churn normalisation; merge success does not reflect quality	Merged PRs only; bug fixes only
Sawada et al. (2026)	Agent PRs in AIDev	Post-merge maintenance	Humans do most maintenance; AI files mostly get feature extensions	Maintenance activity, not design structure
Xu et al. (2025)	Copilot users	Productivity and review load	Core developers review 6.5% more code, own output falls 19%	Organisational study; no design metrics

2.5 Critical Analysis and Motivation for This Work

The studies reviewed above show that coding agents are well evaluated on function-level and issue-level tasks, and that structural problems in generated code are real. They also leave three limits, which motivate the experiment of this thesis.

Real contributions at scale. Existing evaluations either test functions and single issues (HumanEval, SWE-bench) or assess design on small sets of tasks (ProjDevBench's 20 problems, NITR's 21 probes) or on newly generated projects, and most of the studies use a single language. Few look at what agents actually contribute to existing repositories, including contributions that maintainers rejected or never decided on.

Unmeasured validity of the detectors. Many studies assume that a metric or a tool output captures a design principle. Language-model studies report accuracy on curated examples, and tool-based studies report counts of issues without checking that each issue is a violation of a given principle. How far a structural detector agrees with a reading of the same code is rarely measured.

Agent-level comparison. Much of the literature varies the model and treats the agent software as fixed. Ben Sghaier et al. (2026) show that the agent software affects quality with the model held fixed, but whether this reaches design quality has not been compared per principle across several agents on a shared corpus. The studies built on the AIDev corpus look at merge rates, issues in merged bug fixes and post-merge maintenance, not at SOLID adherence.

This thesis therefore runs an experiment that addresses these limits. It measures adherence to the five SOLID principles in real pull requests written by three coding agents, taken from AIDev, in two languages, Java and Python, and it measures how far each detector can be trusted. Chapter 3 describes the design of this experiment.

Chapter

3

Research Methodology

3.1 Research Design Overview

This study is a quantitative empirical benchmark of how well code written by AI coding agents follows object-oriented design principles, in Java and in Python. The unit of analysis is the pull request: one change an agent submitted to a real project, whether or not it was merged. The object of measurement is adherence to the five SOLID principles, assessed one principle at a time with deterministic tools and then checked against an independent reading of the code.

The study has two parts. Chapter 2 reviews work on AI code generation, coding agents and software quality measurement, and identifies the gap the thesis addresses. This chapter describes the empirical design, and Chapter 4 applies it to two corpora of agent pull requests, one in Java and one in Python, drawn from the AIDev dataset (Li et al., 2025).

Four design decisions define the study. They are justified below and in Section 3.5.

1. **Real pull requests, not newly generated repositories.** Running agents to generate many new codebases would be costly and would produce artificial tasks. Measuring code the agents actually submitted to projects gives up control over the tasks but gains realism, and each pull request is a finished change that can be attributed to one agent.

2. **All three outcomes.** A merged pull request has passed review, so a corpus of merged requests measures the combined output of the agent and its reviewers. The corpus therefore contains merged, rejected (closed without merging) and open requests, with rejected and open requests over-sampled relative to the dataset. This makes it possible to examine what review filters out.

   A rejected request is not necessarily a request refused for its quality. Only requests with an explicit request for changes (`changes_requested`) or a completed review followed by closure (`reviewed_then_closed`) record a judgement about the code; most closed requests were superseded, went stale or were abandoned. The outcome comparison is therefore always reported by these strata.

3. **Two languages, Java and Python.** SOLID describes object-oriented design, and one can argue that measuring it in a language that does not require objects says more about the language than about the agent. Java removes that objection: classes are compulsory, `interface` is a keyword, `@Override` is explicit and subtyping is nominal. The reverse objection is that a result found in Java may only describe Java. The study is therefore carried out in both languages. Because the framework is parameterised by language (Section 3.3.7), the whole study runs on Java and on

Python, where classes are optional and no compiler checks a contract. A result is called general only if both languages show it.

4. **Deterministic detectors, validated by a reading judge.** Every reported rate is computed by a program that gives the same output on every run. A large language model is used only to read source code and label a validation sample; it never computes a reported rate. Every agreement figure is therefore agreement between a detector and the judge (Sections 3.3.4 and 4.7.1).

The design is cross-sectional for each pull request. For Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion it compares the repository immediately before and immediately after the change (Section 3.3.3). For Single Responsibility it evaluates the touched classes in the tree after the change. It does not follow an agent across a sequence of changes.

3.1.1 Research Paradigm

The study takes a post-positivist position. It looks for measurable and reproducible signals of design quality while accepting that no static tool fully captures a semantic property such as responsibility or substitutability. Each measurement is treated as a proxy for the underlying principle, and the gap between proxy and principle is quantified with precision and recall against the reading judge.

3.1.2 Study Type

The study is observational. The agents worked on their own tasks in repositories this study does not control, before anything was measured. No treatment was assigned, so the design supports comparison and description but not causal claims (Table 3.1).

3.2 Research Questions

The study answers the two research questions of Section 1.3. Table 3.2 summarises how each is answered, and the paragraphs below describe the analyses behind it.

RQ1, adherence and differentiation. *Do coding agents differ in how well their pull requests adhere to the five SOLID principles, and how many of the design problems in the code they touch do they introduce themselves, in Java and in Python?*

RQ1 is answered by four analyses. The first three are carried out on the Java corpus and on the Python corpus, and the fourth compares the two.

Differences between agents. On a common corpus, with repository and change size controlled, do agents leave different structural footprints, and on which principles?

Table 3.1: Study type.

Dimension	Description
Purpose	Comparison of agents and validation of the measurement instruments
Unit of analysis	Pull request, with class-level and module-level detail
Subjects	Three AI coding agents: Claude Code, GitHub Copilot, OpenAI Codex
Outcomes	Merged, rejected (closed without merging) and open, with rejected and open over-sampled
Languages	Java and Python, with the same framework in both
Data source	AIDev dataset (Li et al., 2025), curated subset
Data collection	Automated static analysis of the source before and after each pull request
Analysis	Per-agent and per-outcome rates (pooled and averaged over repositories), a joint regression model, and validation precision and recall

Merged and unmerged requests. A corpus of merged pull requests shows what maintainers accepted, which is not the same as what agents produced. If the code in rejected requests were structurally worse, the merge decision would be filtering on design quality, and a study of merged code would attribute that filtering to the agents. Including rejected and open requests makes the comparison possible.

Locus of risk. A class or a method can be written in full by one pull request. A dependency cycle, an interface and its implementers, or a base class and its override involve at least two units, and one pull request rarely writes both sides. Single Responsibility and Open/Closed violations lie inside one unit; Liskov Substitution, Interface Segregation and Dependency Inversion violations lie in a relation between units (Figure 3.1). The analysis compares the violations already present in the code a pull request touched with the violations the pull request introduced. The answer indicates which kinds of design damage a single contribution can cause and which kinds build up over many contributions.

Both languages. Each answer could depend on the language. Change size could dominate because Java code is verbose, an agent difference could reflect how one agent writes Java, and relation-level violations could be rare because the Java compiler rejects some of them. The second language tests these possibilities. A second corpus is drawn from the same dataset for the same agents and outcomes. The fact extractor and the thresholds are replaced; the detector logic, validation procedure, judging rubric and statistical model are reused. An answer to RQ1 is called general only if both languages show it.

Figure 3.1 shows the distinction behind the locus of risk. The dashed gold area is what one pull request changes. On the left the whole class is inside the change, so the change can create the violation. On the right the violation lies on the relation between

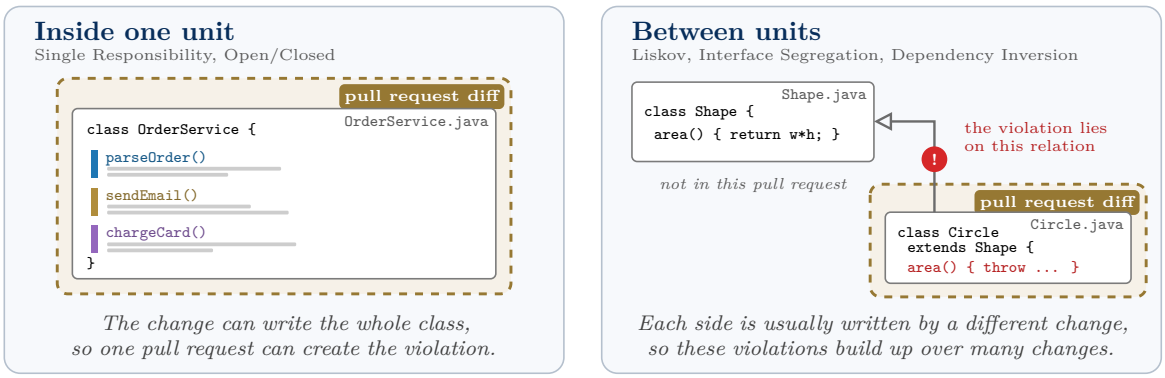


Figure 3.1: Intra-unit and inter-unit violations.

two files, and the change covers only one of them.

RQ2, measurability and construct validity. *Can SOLID adherence be measured at repository scale with deterministic structural detectors, and how well do these detectors agree with an independent reading judge and with established analysers run on the same code?*

SOLID principles concern meaning and intent, while static metrics describe structure. RQ2 asks how far a structural detector can be trusted, and answers with precision and recall per principle against labels from a judge that reads the source, in both languages. For Java, a second check compares the class-level detectors with three published analysers, CK, PMD and DesigniteJava, run on the same source trees (Section 4.6.1). The two checks can fail in different ways: the judge can err in the same direction as a detector, while an independently built tool has no access to the detector's rules.

Table 3.2: Research questions and how each is answered.

RQ	Focus	Construct	Method
RQ1	Agents, outcomes, locus of risk, two languages	SOLID adherence per principle	Rates per agent and outcome, pooled and per repository; joint regression model; violations present compared with violations introduced; 4,245 Java and 4,946 Python pull requests
RQ2	Construct validity	Detector fidelity	Precision and weighted recall on 1,495 judged Java and 980 judged Python entities, including a Java holdout sample; agreement of the Java detectors with three external analysers

3.3 Research Strategy

3.3.1 A Benchmark over Real Pull Requests

A fixed corpus of agent pull requests is processed by one deterministic analysis procedure, and the results are compared across agents. Mining finished work has two advantages: the code was written for a real task in a real project, and each pull request can be attributed to one agent. The cost is that task difficulty cannot be held constant across agents. Repository controls (Section 3.3.6) and the joint model reduce this problem, and Section 5.6 states what remains.

The corpus comes from the AIDev-7.6M release, whose master tables hold 7,685,281 agent pull requests in 957,209 repositories. The agents are very unevenly represented: OpenAI Codex has 3,844,602 pull requests and Devin 77,791. The working corpus is therefore sampled by agent and outcome; Section 4.2 describes the curation.

3.3.2 One Deterministic Detector per Principle

Each principle is implemented as a pure function over extracted facts. The function receives the facts for a source tree and returns a structural measurement and a flag. It does not depend on how the source was obtained or on any external service, so the same input always gives the same output. The Single Responsibility detector runs on the touched classes, and the other four run on complete before and after trees through the same interface. Section 3.4 summarises each detector and Chapter 4 gives the exact rules and thresholds.

3.3.3 The Before/After Delta

For Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion, a single snapshot of the repository cannot show what the agent changed. The analysis therefore reconstructs the repository immediately before and immediately after the pull request, analyses both trees, and charges the pull request only with the difference: a new dependency cycle, a newly widened interface or a newly incompatible override. For a merged request the two trees come from the merge commit and its first parent; for an unmerged request they come from the request's branch point and head (Section 4.2).

Figure 3.2 illustrates the delta for Dependency Inversion. The same analysis runs on the tree before and after a pull request. In the example the pull request edits modules `a` and `c` (gold rings) and adds the import `c`→`a`, which makes three module pairs mutually reachable; only this difference is charged to the pull request.

Two attribution rules change what is measured. First, classes an agent created are scored separately from existing classes it edited. Scoring only the touched classes would

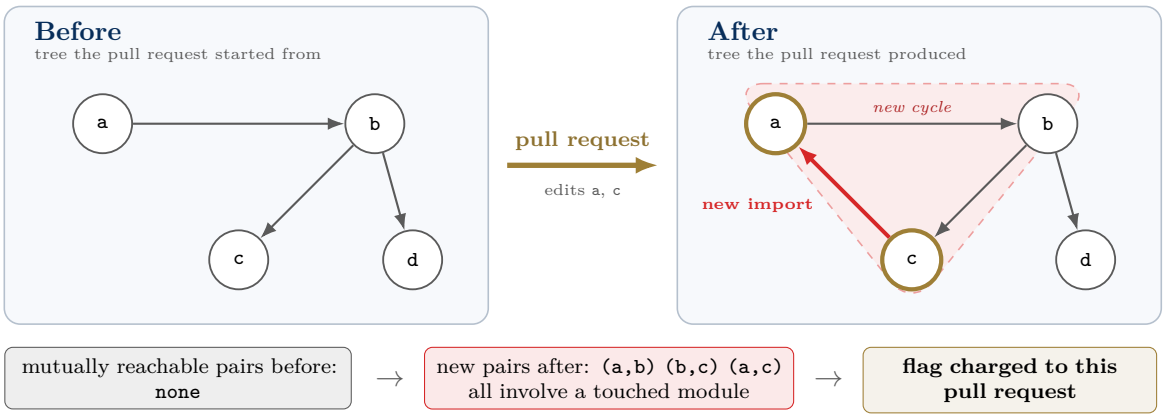


Figure 3.2: The before/after delta for Dependency Inversion.

charge the agent with the debt already in the repository (Section 4.7.5, Finding 4). Second, for the relation-level principles a violation is charged when either side of the relation is in a touched file. An agent that edits a base class or widens an interface is therefore charged for the subclasses and implementers it breaks in files it did not open.

For ISP, LSP and most DIP cases, a single pull request rarely creates a whole violation, because the violation spans code the request did not write. Each of these detectors therefore also records an inventory of the relevant structures at the head of the pull request (every interface, override pair or dependency edge), reported separately from the per-request delta.

3.3.4 Validation by a Reading Judge

For each principle a stratified sample of flagged and unflagged items is drawn. A large language model of the Claude family acts as the judge: it reads the source of each item and labels it as a violation or not, with a written reason. For LSP the judge sees the overriding method next to its base method. The detector's flag is then scored against these labels, so every precision and recall figure in this thesis is agreement between the detector and the judge. The label columns are named accordingly (for example `llm_srp_risk`). The judge sees the entity and its source but not the detector's verdict.

3.3.5 Screens, Suppressors and Refinements

The Single Responsibility and Open/Closed detectors are designed as screens: their thresholds aim to miss few violations and accept false alarms. Each has a suppressor that removes one named class of false positives. For Single Responsibility, a class whose methods share a common vocabulary (conceptual cohesion, C3) is not flagged. For Open/Closed, a dispatch over an enumeration is not flagged, because the language fixes its set of values. Whether the screens reach high recall is measured in Section 4.7.1.

A change to a detector was accepted only if the reason it should work could be stated

in advance. A restriction derived from reading false positives can be justified by the rows it removes. A threshold found by trying values until the score improves cannot be distinguished from fitting noise.

Two refinements reported in Chapter 4 were derived by reading the false positives of a first sample. Scoring them on the same sample would overstate their performance, so each was scored again on a holdout: 200 further entities drawn with a different seed, excluding every row judged before, and judged before the two rounds were compared.

3.3.6 Controlling for Repository Mix

Because the corpus is mined, an agent's pooled rate can be dominated by one large repository in which it happens to work often. Every comparative rate is therefore reported twice: pooled over all of an agent's units, and as the mean of per-repository rates (the macro average). The share of an agent's pull requests that come from its largest repository is reported as a concentration check. Where the two rates disagree, the macro average is preferred.

3.3.7 Language Selection

Three detectors work on classes: the God Class rubric (SRP), the interface width and stub rule (ISP) and the override comparison (LSP). They can only measure an agent if the corpus contains enough classes.

Java meets this condition by design. Every Java compilation unit declares a type, so every pull request that changes Java source touches a class. Each principle also has a direct counterpart in the syntax: `interface` declares a contract, `@Override` marks an override, subtyping is nominal, and the compiler rejects several violations that other languages allow at run time.

The last property has a consequence developed in Chapter 4. Three of the signals a Liskov detector needs in a dynamically typed language cannot occur in Java code that compiles: a changed parameter list makes an overload, parameter types are invariant, and visibility cannot be reduced. The Java detector keeps only the signals the compiler does not check.

Only the fact extractor depends on the language. It turns source code into a language-neutral record of types, members, supertypes, imports and member accesses, and every detector, calibration step, validation procedure and report reads that record. A second language therefore needs a new extractor, not a new study.

Python was chosen as the second language because it lacks the properties that made Java convenient. A Python module does not need to declare a class; a contract can be an `abc.ABC`, a `typing.Protocol` or a base class whose methods raise `NotImplementedError`;

and nothing checks an override against its base before the program runs. The class precondition is met in practice: the analysable Python pull requests touch 81,149 production classes. The compiler argument also makes a testable prediction. If `javac` removes a family of Liskov faults, those faults should make up a large share of the Liskov violations a judge confirms in Python. Section 4.8.3 reports that they do.

3.4 Mapping SOLID to Structural Detectors

This section describes, at the level of method, how each principle is turned into a computable detector. Chapter 4 gives the exact thresholds, and Table 3.3 summarises the mapping.

SRP (Single Responsibility).

The targeted smell is the God Class, a class doing several unrelated jobs. The detector combines a six-metric rubric of size, complexity and cohesion (class lines of code, method count, weighted methods, per-method complexity and parameters, and LCOM) with conceptual cohesion, C3 (Marcus et al., 2008). A class is flagged when it crosses at least two thresholds and its methods do not share a common vocabulary. Size alone cannot separate a large class with one topic from a large class with several, which is why cohesion is combined with size.

OCP (Open/Closed).

The targeted smell is a type or tag switch that must be edited whenever a new variant is added. The detector finds dispatch chains over the type or tag of one subject (`instanceof` ladders, `getClass()` comparisons and `switch` statements on an enumeration or a tag value), flags chains longer than a calibrated number of branches, and suppresses chains over a set of values fixed by the language. Comparing the before and after trees shows whether a pull request extended such a chain or added a new one.

LSP (Liskov Substitution).

The targeted smell is an override that cannot stand in for its base: an inherited method that is refused, a precondition strengthened by a new argument check, or a postcondition weakened by returning `null` where the base never does. In Java the compiler already rejects signature faults (a changed parameter list is an overload, parameter types are invariant, and visibility cannot be reduced), so the Java detector looks only for faults the compiler does not check. It works over the resolved type hierarchy, which avoids building each of the 1,010 repositories. The Python detector adds the signature signals (Section 4.8.2).

ISP (Interface Segregation).

The targeted smell is a wide interface whose implementers must stub methods they do not need. The detector records the width of every contract (its number of abstract members) and, for every implementer, the share of members it stubs. A pull request is flagged when it creates or widens a contract beyond a calibrated width, or leaves an implementer with forced stubs. Width is a screen: a wide interface whose implementers use every member is not a violation. The forced stub is direct evidence.

DIP (Dependency Inversion).

The targeted smell is a module that depends on concrete details, and in particular a dependency cycle. The detector builds the internal import graph before and after the pull request, finds strongly connected components with Tarjan's algorithm (Tarjan, 1972), and flags a change that makes two modules newly reachable from each other. It also records the change in mean distance from Martin's main sequence and whether new dependencies point to abstract or concrete modules, as description only. The distance signal confirmed none of its flags when validated, so it does not enter the flag (Section 4.4.6).

Component zones (descriptive only).

A sixth evaluator places every package on Martin's abstractness and instability plane (Martin, 2003): the zone of pain (concrete and stable), the zone of uselessness (abstract and unused) and the main sequence between them. It uses the A, I and D values the DIP detector computes and raises no flags. A package with no classes or no internal dependencies has undefined coordinates and is reported as unplaceable, not as zero; recording zero would place every class-free utility package in the zone of pain.

3.5 Validity

This section lists the design decisions that protect validity. The threats that remain are collected in Section 5.6.

Internal validity. Every measurement is produced by a deterministic program from source text, so every reported number can be regenerated from the corpus. Thresholds are calibrated from percentiles of each corpus before any labelling, so a detector is never tuned on the sample used to score it. A violation is charged to a pull request only through a before and after comparison, so a violation that already existed is not charged to an agent that edited nearby code.

Table 3.3: From principle to detector (details in Chapter 4).

Principle	Smell	Detector	Main metric or rule
SRP	God Class	size and complexity rubric with a cohesion suppressor	LCOM; C3 conceptual cohesion
OCP	type or tag switch	dispatch-chain length with a closed-set suppressor	branch count; closed-set test
LSP	incompatible override	faults the compiler does not check, over the resolved hierarchy	refused bequest; argument checks; returning null
ISP	wide interface, forced stubs	contract width and implementer stub ratio	abstract-member count; stub ratio
DIP	dependency cycle	import graph before and after	new cyclic module pairs (Tarjan)
Zones	descriptive only	package position on the A/I plane	pain, main sequence, uselessness, intermediate, unplaceable

Construct validity. Each detector is validated on a stratified sample of flagged and unflagged entities. A sample of flagged entities alone can estimate precision but cannot find missed violations. Every label is written by a judge that reads the entity's source and records a reason. For Java, the class-level detectors are also compared with three independently developed analysers (Section 4.6.1).

Statistical validity. Counts are reported next to rates, and proportions carry Wilson score intervals, which behave well for small counts. Rates are reported pooled and as macro averages over repositories. The joint model estimates agent effects with standard errors clustered by repository, so many pull requests from one repository are not treated as independent. Because several agent coefficients are tested, each is also compared with a Bonferroni threshold over the agent coefficients of its language (Section 4.7.3).

External validity. A result from one language may describe the language and not the agents. The study addresses this by repeating corpus construction, calibration, validation and modelling in Python with the same agents (Section 4.8). Each language's thresholds are calibrated on its own corpus before its labels exist.

Reliability. The pipeline can be resumed after interruption, facts are cached by the content hash of the source tree, and each pure module has a self-check that runs as a script. A mechanical audit checks that every judged row traces to the source file the judge was shown.

3.6 Ethical Considerations

The study involves no human participants and no personal or proprietary data. Pull request metadata comes from the public AIDev dataset. The repositories were public and carried a permissive licence at the time of collection; they were cloned locally for analysis and are not redistributed. No model was trained. The only use of a language model is as a reading judge on public source code during validation.

Chapter

4

Experimental Setup and Results

4.1 Overview

This chapter applies the methodology of Chapter 3. It describes the corpus (Section 4.2), the tools (Section 4.3), the exact detector definitions (Section 4.4), the pipeline (Section 4.5) and the validation procedure (Section 4.6). Section 4.7 reports the Java results: detector validation, agent differences, the locus of risk and the cross-cutting findings. Section 4.8 reports the results in Python. Limitations are collected in Section 5.6.

The analysis code is written in Python and is deterministic. It needs `pandas`, `pyarrow`, `statsmodels` and the `git` client, plus a JDK for the Java fact extractor; the Python fact extractor uses only the standard library. No step that computes a reported number depends on a network service or a language model.

4.2 Dataset

4.2.1 Source

The corpus is drawn from AIDev (Li et al., 2025), a collection of pull requests opened by autonomous coding agents on public GitHub repositories, with repository and commit metadata. Two properties of the release shaped the curation.

First, the released tables describe different populations. `pull_request.parquet` and `repository.parquet` contain only a subset of repositories with more than 100 stars. The complete tables are `all_pull_request.parquet` and `all_repository.parquet`, and all figures in this thesis come from them.

Second, file-level attribution is missing for some pull requests, and which agent is most affected changes between releases. The curation therefore treats every agent the same way: when the dataset has no file list for a pull request, the list is retrieved from the GitHub API.

4.2.2 Curation

Curation keeps pull requests that change Java source files, identified by file extension, so that Java modules inside repositories with another primary language are included. Repositories must have at least 25 stars and a permissive licence (MIT, Apache-2.0, BSD-2-Clause or BSD-3-Clause, ISC, 0BSD, Unlicense, CC0-1.0 or MPL-2.0).

The outcomes are merged, rejected (closed without merging) and open (undecided when the data was collected). Within each agent, every eligible rejected and open pull request is included, and merged pull requests are drawn at twice the number of rejected ones. Table 4.1 shows the result, with the number of repositories in each cell and the share of its pull requests that come from its largest one (top-1) and its largest three

(top-3) repositories.

Table 4.1: Java corpus by agent and outcome.

Agent	Outcome	PRs	Repos	top-1	top-3
Claude Code	merged	970	271	0.9%	2.8%
Claude Code	rejected	485	171	15.1%	23.9%
Claude Code	open	283	137	7.1%	20.1%
GitHub Copilot	merged	2,544	372	7.6%	22.7%
GitHub Copilot	rejected	1,272	334	9.7%	21.4%
GitHub Copilot	open	364	149	10.2%	23.4%
OpenAI Codex	merged	630	122	4.0%	11.7%
OpenAI Codex	rejected	315	80	13.3%	36.5%
OpenAI Codex	open	93	57	17.2%	26.9%
Total		**6,956**	**1,010**		

No cell takes more than 17.2% of its pull requests from a single repository. Because one large repository could still dominate an agent's rate, every rate is reported both pooled and as a macro average over repositories.

The corpus is not representative. Over-sampling rejected and open requests gives the outcome comparison its statistical power, but it does not match the world. Across the 7,685,281 agent pull requests in the dataset, 80.9% are merged, 9.5% are closed without merging and 9.6% are still open. Every per-outcome rate in this chapter is therefore valid as printed, while any rate pooled across outcomes describes this sampling design and not the population. Reading a pooled rate as a population rate would need it reweighted to those shares.

Rejected requests. Table 4.2 splits the requests that were not merged by how they ended. Few carry an explicit objection. Most were closed without any review, which in practice means they were superseded, went stale or were abandoned. `changes_requested` marks a recorded request for changes, and `reviewed_then_closed` a completed review followed by closure. The outcome comparison is reported by these strata.

Table 4.2: How the Java pull requests that were not merged ended.

Agent	changes_ requested	reviewed_ then_closed	closed_ unreviewed	open
Claude Code	1	11	473	283
GitHub Copilot	42	169	1,061	364
OpenAI Codex	2	36	277	93

4.2.3 Agent Selection

The same three agents, Claude Code, GitHub Copilot and OpenAI Codex, are used in both languages, so that every cross-language comparison has the same agents on both sides. Cursor and Devin were excluded because they have too little Java: 94 eligible Java pull requests for Cursor, 72.3% of them from three repositories, and 6 for Devin. Google Jules was excluded because it appears for the first time in this release of the dataset, so its attribution of pull requests has no earlier release to be checked against.

4.2.4 Reconstruction and Attrition

Measuring what a pull request changed requires the source tree before and after it. The pipeline tries four methods in order: (1) the merge commit and its first parent; (2) the pull request's own commits; (3) the `refs/pull/N/head` reference and its merge base; (4) the base and head commits reported by the GitHub API. The first method exists only for merged requests, so the comparison between merged and unmerged code has to control for the method used. Table 4.3 accounts for every curated pull request.

Table 4.3: From the curated Java corpus to the analysable corpus.

Status	PRs	Meaning
`ok`	4,245	analysable: valid before/after pair, Java changed
`no_touched_files`	2,540	no production Java file changed
`repo_too_large`	150	source tree above the parsing limit
`archive_failed`	18	history rewritten or repository deleted
`no_delta_pair`	3	no method recovered a distinct before tree
Total	6,956	

The attrition affects the outcome comparison in two ways. The analysable share differs by outcome: 62.1% of merged requests, 66.9% of open requests and 56.8% of rejected

requests. The reconstruction method also differs, because method 1 is available only for merged requests. The outcome comparison is therefore made on a tier-matched subset (methods 2, 3 and 4), where both sides recover their before tree in the same way.

The exclusions are not random. The 150 requests dropped as `repo_too_large` come from the largest repositories, where design problems have the most room to accumulate. The 2,540 requests that changed no production Java lean towards configuration and documentation work. The same filters apply to all three agents, so they do not bias the comparison between agents, but the rates in this chapter describe repositories within the parsing limit.

An empty file list is accepted only when the GitHub API confirms it. An empty diff caused by a retrieval failure is recorded as a failure, not as a request that changed nothing.

4.2.5 Resolution Quality

The Java extractor resolves supertypes and imports from source alone, without building the project, so types defined outside the repository cannot be resolved. The median share of resolved supertypes per pull request is 87.0% (10th percentile 37.8%, 90th percentile 98.4%), and 498 touched files failed to parse. Resolution is similar across agents: Claude Code 86.2%, Copilot 85.9% and OpenAI Codex 89.3%.

Resolution falls below 50% in 718 of the 4,245 pull requests, mostly in projects whose supertypes come from frameworks such as Spring, Android or Jakarta. The Liskov and Interface Segregation detectors, which need resolved hierarchies, undercount in these projects. An unresolved supertype removes a candidate; it cannot create a false one. The affected rates are therefore lower bounds.

4.3 Toolchain

4.3.1 Selection

The tools had to run locally, for free, deterministically and without a GPU, so that the benchmark can be repeated over about a thousand repositories. Many mature Java analysers need compiled code. ArchUnit and `jdeps` read class files, SonarQube's Java analysis uses compiled classes, and Error Prone runs inside the compiler. Building 1,010 repositories at the exact commit of each pull request would require resolving historical dependencies for each one, which is slow, not reproducible, and fails more often for older and larger projects.

The analysis is therefore source-only. Java facts are extracted with JavaParser and its symbol solver, which resolves types from the source tree, and the five detectors are pure functions over those facts. The cost is incomplete resolution (Section 4.2.5).

4.3.2 Tools Used

Table 4.4 lists the analysis tools, the role of each and the principles or sections that use it.

The language model used as the label judge (Zheng et al., 2023) is not part of the pipeline. It reads source code and writes labels; every number in this chapter is computed by a deterministic program from those labels and the result files.

Symbol resolution. A syntax-only parser such as tree-sitter is cheaper but not sufficient here. It parses Java but does not resolve names, so two classes called `Node` in different packages become one, and every relation built on them is wrong. In an earlier pilot built on tree-sitter, the Interface Segregation detector found no contracts and the Liskov detector found seven override pairs. Single Responsibility and Open/Closed are local properties that syntax can decide, but Liskov Substitution, Interface Segregation and Dependency Inversion are relations between types and require knowing which type is which.

JavaParser's type solver maps package `a.b.C` to the file `<root>/a/b/C.java`, so it must be given the source root, not the repository root. On the Chronicle-Wire repository, rooting it at the repository root lowered supertype resolution from 93% to 17%. The extractor therefore infers source roots from each file's `package` declaration, which works for Maven, Gradle, multi-module and flat layouts without a build file.

Coverage of existing tools. CK, PMD and DesigniteJava all have size or God Class rules that relate to Single Responsibility, and DesigniteJava also reports hierarchy smells related to Liskov Substitution and cyclic dependencies related to Dependency Inversion. DesigniteJava's *Missing Hierarchy* and *Unexploited Encapsulation* smells address conditional logic on types, which is close to the Open/Closed detector; Section 4.7.2 compares them. None of the three tools has a smell or rule for Interface Segregation.

Deciding whether a dispatch chain ranges over an open set of variants, or whether an interface forces its implementers to write stubs, requires linking declarations across files. Single-file linters do not build such a graph, and compiler-based tools obtain it from a classpath. The source-only extractor provides it without a build, which is what makes the OCP and ISP detectors possible here.

4.3.3 Experimental Environment

All analysis ran on one laptop, without a cluster, cloud computing or GPU (Table 4.5).

The hardware shaped four decisions.

- Compiling the corpus would mean building two commits for each of 4,245 pull requests, 8,490 builds, each with the right JDK and historical dependencies. This

Table 4.4: Analysis toolchain.

Tool	Role	Type	Used by
JavaParser and SymbolSolver 3.25.10	Java parse trees with resolved symbols, written as a language-neutral fact record	Static	all five (Java)
Python `ast` (standard library)	Python parse trees written as the same fact record; supertypes and imports resolved by module path inside the repository	Static	all five (Python)
TF-IDF and cosine similarity (own code)	C3 conceptual cohesion	Information retrieval	SRP
`git`	Before and after source trees of each pull request	Version control	OCP, LSP, ISP, DIP
GitHub REST and GraphQL API	File lists and base and head commits missing from the dataset	Web API	corpus construction
CK 0.7.0	Per-class metrics for convergent validity	Cross-check	Section 4.6.1
PMD 7.7.0	Design rules (`GodClass`, `TooManyMethods`, `NcssCount`, `CouplingBetweenObjects`)	Cross-check	Section 4.6.1
DesigniteJava 1.1.2	Design smells (Suryanarayana et al., 2014)	Cross-check	Section 4.6.1
statsmodels 0.15.0	Generalised estimating equations for the joint model	Statistics	Section 4.7.3

Table 4.5: Machine and software environment.

Component	Specification
Machine	Lenovo IdeaPad Gaming 3, model 82K2
CPU	AMD Ryzen 5 5600H, 6 cores, 12 threads
Memory	16 GB (15.4 GB usable)
Storage	1 TB and 512 GB NVMe SSDs
Operating system	Windows 11 Home Single Language, build 26200
Python	3.11.9, pandas 3.0.5, pyarrow 25.0.1, statsmodels 0.15.0
Java	OpenJDK 17.0.20.1, Maven 3.9.16
Version control	Git 2.55.0

was not feasible, and the failures would not be random. Every Java tool in Table 4.4 runs on a bare source tree.

- Extracted facts are cached by the content hash of the source tree, so a repeated run over the corpus does not parse unchanged trees again.
- DesigniteJava runs with a 5 GB heap, which the machine cannot supply to several instances at once, so it runs one repository at a time while the lighter tools run in parallel.
- Downloads, not processing, limit data acquisition. Repositories are fetched only to a depth of two commits around the merge commit, source trees are read with `git archive` instead of a checkout, and a compressed tree cache per repository and commit is shared by all detectors.

4.3.4 Research and Writing Tools

Table 4.6 lists the software used to conduct and write the research. None of it computes a reported number.

GitHub has two unrelated roles in this study: it hosts this project's code, and the corpus consists of pull requests on public GitHub repositories, taken from the AIDev dataset.

4.4 Detectors, Thresholds and Equations

Each detector is a pure function over extracted facts. This section gives the exact rules. All thresholds are defined in one configuration module and were set from each corpus's own distribution before the corresponding labels were collected.

None of the five principles can be observed directly. Each concerns intent: the reasons a module may change, the variants a design expects, the promises a subtype makes. A

Table 4.6: Research and writing tools.

Logo	Tool	Use
	Zotero	Reference manager for collecting sources.
	MiKTeX 25.12	TEX distribution used to compile the thesis and the defence presentation with XeTeX 4.16; XeTeX is needed for `fontspec` and the Arabic abstract, and BibTEX processes the bibliography.
	`latexmk` 4.88, Perl 5.42	Build driver that reruns XeTeX and BibTEX until cross-references are stable; `latexmk` is a Perl program.
	VS Code 1.137	Editor for the pipeline, the analysis scripts and the thesis.
	Git and GitHub	Version control and hosting for the pipeline, its configuration, the result tables and the thesis source.

static detector can only observe a structure that a violation tends to leave behind. As Table 4.7 shows, each principle asks for something different from what is computed instead; the table also gives the unit of measurement and the agreement of each detector with the reading judge (precision / weighted recall, Section 4.7.1). Every result in Section 4.7 is first a statement about the detector and only through it a statement about the principle.

Two points govern how the table should be read. Each detector reacts to a structure that violations usually produce, so a violation without that structure is invisible to it, and its recall is limited by the choice of structure as much as by the threshold. And each agreement figure describes how often a flag matches a judge reading the same source, which is agreement with a reading of the principle, not with ground truth.

Attribution for relation-level principles. For LSP, ISP and DIP, a violation is charged to a pull request when either side of the relation is in a touched file. Charging only the flagged entity would miss an agent that edits a base class or widens an interface and so breaks subclasses or implementers in files it did not open. With this rule, 81,235 override pairs and 20,705 contracts are in scope across the Java corpus. The rule changes which pull request a violation is charged to, not how many violations exist.

4.4.1 SRP: God Class Rubric with a Cohesion Suppressor

A class receives one rubric hit for each of six thresholds it reaches: class length 362 lines, weighted methods per class 45, 20 methods, a method with cyclomatic complexity 11, a method with 5 parameters, and LCOM 0.86 (for classes with at least four methods). Each threshold is the 90th percentile of the distinct production classes in the Java corpus. A class is flagged (`srp_risk`) with at least two hits, and `srp_risk_strict` marks classes

Table 4.7: What each principle asks and what the Java detector computes.

Principle	What it asks	What is computed	Unit	P / R
SRP	A class should have one reason to change	Six-metric God Class rubric (class length, weighted methods, method count, largest method complexity, largest parameter count, LCOM) with a C3 conceptual cohesion suppressor	Class	0.350 / 0.828
OCP	Open for extension, closed for modification	Length of type and tag dispatch chains (`instanceof` ladders, `switch` on an enumeration or a tag), with a suppressor for enumerations	Dispatch chain	0.330 / 0.742
LSP	A subtype must be usable in place of its base	Override compared with its nearest ancestor over the resolved hierarchy: refused bequest, strengthened precondition, weakened postcondition	Override pair	0.420 / 0.386[a]
ISP	Clients should not depend on members they do not use	Contract width and implementer stub ratio for every interface and abstract class	Contract	0.335 / 0.464[b]
DIP	Depend on abstractions, not concretions	Module pairs that become mutually reachable in the import graph, charged when one side is a changed file	Pull request	0.892 / 1.000[c]
Zones	(description)	Package position on Martin's A/I plane	Package	not validated

[a] Restricted to refused bequest, precision is 1.000 on 64 flags. [b] Restricted to contracts of 30 or more members, precision is 0.786. [c] The flag uses new cycles only; the change in distance from the main sequence is recorded but does not flag (Section 4.4.6).

with three or more.

Cohesion (LCOM). Following LCOM4 (Hitz and Montazeri, 1995), the detector builds a graph with one node per method and an edge between two methods that use a common attribute or where one calls the other. Constructors are excluded, because a constructor uses every attribute and would join unrelated groups of methods into one component; in Python the same rule removes `__init__`, `__new__` and `__post_init__`. With c connected components and n methods, LCOM is normalised as

$$\mathrm{LCOM} = \frac{c-1}{n-1}, \tag{4.1}$$

so 0 means one connected group and 1 means that no two methods are connected. On a synthetic class with three separate responsibilities, the version that kept constructors scored LCOM = 0; with constructors excluded it scores the class as split.

LCOM is left undefined when the method graph can have no edges for syntactic reasons, for example an interface whose methods have no bodies. Scoring such types would add a cohesion hit to types whose only real signal is their number of methods. The test is made on the graph and not on the keyword, so an interface whose default methods call one another keeps its value.

Conceptual cohesion (C3). Following Marcus et al. (2008), each non-constructor method is treated as a bag of identifier and comment terms, each bag becomes a TF-IDF vector, and C3 is the mean cosine similarity over all pairs of methods:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|} \tag{4.2}$$

$$C3 = \operatorname*{mean}_{i<j} \cos(v_i, v_j) \tag{4.3}$$

where v_i is the TF-IDF vector of method i. C3 is near 0 when methods share no vocabulary and near 1 when all methods use the same terms. A class with exactly two hits and $C3 \geq 0.4$ is not flagged, because its methods share one vocabulary. A class with three or more hits is flagged regardless of C3.

4.4.2 OCP: Dispatch Chain Length with a Closed-Set Suppressor

The detector finds every dispatch chain, an `if/else if` ladder or a `switch` that branches on one subject, and classifies it by what it tests: `instanceof` and comparisons with a class literal or `getClass()` are type-based; `switch` on a value and on a tag are tag-based. The Java result files contain 41,880 chain rows with at least two branches.

Thresholds were calibrated on distinct production chains before any label existed.

Type-based chains are flagged at three or more branches. Tag-based chains are flagged at eight or more, the 90th percentile of value switches, because short value ladders are often parsers or state machines.

The closed-set suppressor removes dispatch over a set of values the language fixes:

$$\text{smell} \;=\; (\text{branch count} \geq \text{threshold}) \;\wedge\; \neg\,\text{closed}(\text{subject}),$$

where closed holds when the subject's resolved type is an `enum`. A client cannot add a value to an enumeration, so a switch over one is not the kind of growing conditional that OCP targets. The rule suppresses 6,073 chain rows.

The rule recognises only sets that the language fixes. Many closed sets in this corpus are fixed by external specifications instead, such as JDBC type codes, protobuf scalar types, Unicode blocks and HTTP status classes, and the compiler sees them as open. Section 5.7 proposes recognising them.

4.4.3 LSP: Override Compatibility over the Resolved Hierarchy

In a dynamically typed language a Liskov detector must check that an override keeps its base's signature. In Java the compiler does this: a method with a different parameter list is an overload, parameter types cannot be narrowed, return types can only become more specific, and visibility cannot be reduced. Code that compiles cannot contain these faults, so looking for them would only enlarge the set of pairs examined. The Java detector looks instead at whether the override behaves like its base.

For every pair of an internal subclass method and the nearest ancestor method with the same signature, it computes three signals:

- `refused_bequest`: the base method has a working body and the override only throws `UnsupportedOperationException` or a similar refusal exception;
- `precondition_strengthened`: the override adds argument checks that throw where the base accepted the call;
- `postcondition_weakened`: the override can return `null` where the base never does.

Three exclusions apply. Constructors and static methods are excluded because they are not called through a base-type reference. The five `Object` methods (`equals`, `hashCode`, `toString`, `clone` and `finalize`) are excluded because almost every class declares them. And the base method must have a real body: a pair whose ancestor declares an abstract or interface method has no inherited behaviour to refuse or to strengthen.

Section 4.7.1 shows that the three signals differ sharply in precision, and that restricting the flag to refused bequest is the most effective single change made to any detector in this study.

4.4.4 ISP: Contract Width and Implementer Stub Ratio

In Java a contract is an `interface` or an `abstract class`. The detector indexes both across the whole repository and follows the resolved hierarchy, so members inherited from a super-interface count towards the contract they impose. The Java result files contain 20,705 contract rows.

A contract is considered only if it has at least one abstract member and at least one implementer inside the repository; otherwise every marker interface would count as a contract. For each contract the detector records its width, the number of abstract members, and for each implementer its stub ratio, the share of implemented members whose body is empty or only throws a refusal exception. A contract is wide at 13 or more members, the 90th percentile of distinct contracts; 2,944 contract rows reach this width, and 744 implementer rows carry forced stubs (at least two stubs forming at least half of the implemented members). A pull request is flagged when it creates a wide contract, widens one, or leaves an implementer with forced stubs. Width is a screen: a wide contract whose implementers use every member is not a violation, which is why the stub ratio is recorded alongside it.

4.4.5 DIP: Import Graph Delta

The detector builds the internal import graph of the whole repository before and after the pull request and computes:

- **Cycles**: strongly connected components with more than one module, found with Tarjan's algorithm (Tarjan, 1972). A pair of modules (u, v) is newly cyclic when u and v are mutually reachable after the change but not before, and the pull request is flagged when such a pair involves a touched file. Three definitions of a new cycle were implemented and gave counts that differ by an order of magnitude; all DIP rates in this thesis use this one. Attribution is by compilation unit, not by package. Filtering by package is too coarse for projects that keep most code in one package: in one Android project it admitted 44 newly cyclic pairs, of which only one involved a touched file.

- **Martin metrics** per package: instability $I = C_e/(C_a + C_e)$, abstractness $A = N_a/N_c$ and distance from the main sequence $D = |A + I - 1|$, where C_e and C_a are efferent and afferent couplings and N_a and N_c are the numbers of abstract and total classes (Martin, 2003). The change in mean D is recorded for every pull request but does not flag (Section 4.4.6).

- **Abstract dependency ratio**: the share of a touched module's new dependencies that point to modules defining an abstraction, recorded as context.

4.4.6 Component Zones: a Package-Level Description

The five detectors work below the package. To describe the packages themselves, a sixth evaluator places each package on Martin's abstractness and instability plane, using the metrics defined above. As Figure 4.1 shows, concrete, stable packages fall in the zone of pain and abstract, unused packages in the zone of uselessness, and packages within 0.15 of the line $A + I = 1$ are on the main sequence.

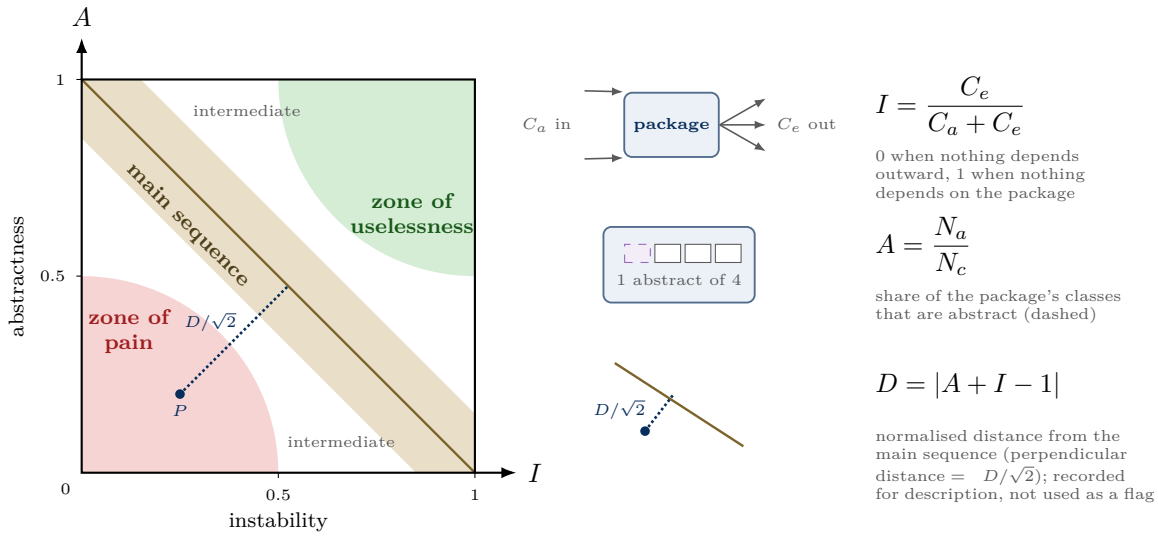


Figure 4.1: Martin's abstractness and instability plane.

- The zone of pain is the corner region of radius 0.5 around $(A{=}0, I{=}0)$: concrete and stable packages that many others depend on and that are therefore hard to change. The zone of uselessness is the region of radius 0.5 around $(1, 1)$: abstract packages that nothing depends on. Packages within 0.15 of the line $A + I = 1$ are on the main sequence, and the rest are intermediate. The radius and the band are presentation choices, stated so that a reader can change them.
- A package is unplaceable when a coordinate is undefined: A for a package with no classes, I for a package with no internal dependencies. Recording these as 0 would place every class-free utility package in the zone of pain.

The evaluator reads only the after tree and raises no flags. The distance from the main sequence was tested as a signal of change in two validation rounds on earlier pilot corpora and was confirmed on none of 17 flags: mean D moves when files are added or moved, even when no dependency changes direction. The position of a package on the plane is still a valid description of that package. Treating a package in the zone of pain as a violation would need its own validation.

Packages are grouped by parent package, not by the first segment of the module name. A reverse-domain prefix would place most of a repository in one package, which gives $C_e = C_a = 0$ and an undefined I. The classifier has a self-check that includes the rule that an undefined coordinate never produces a zone.

4.5 Analysis Pipeline

The pipeline is a sequence of resumable stages. Each stage reads the Parquet outputs of the previous stage and writes its own, so any stage can be rerun alone. As Figure 4.2 shows, the corpus stage samples agents and outcomes, the measurement stage recovers a before/after pair for each pull request and scores all five principles from one parse, and the validation stage checks the detectors against a reading judge and against three external analysers. The figure shows the Java pipeline; the Python study follows the same stages with its own extractor (Appendix A).

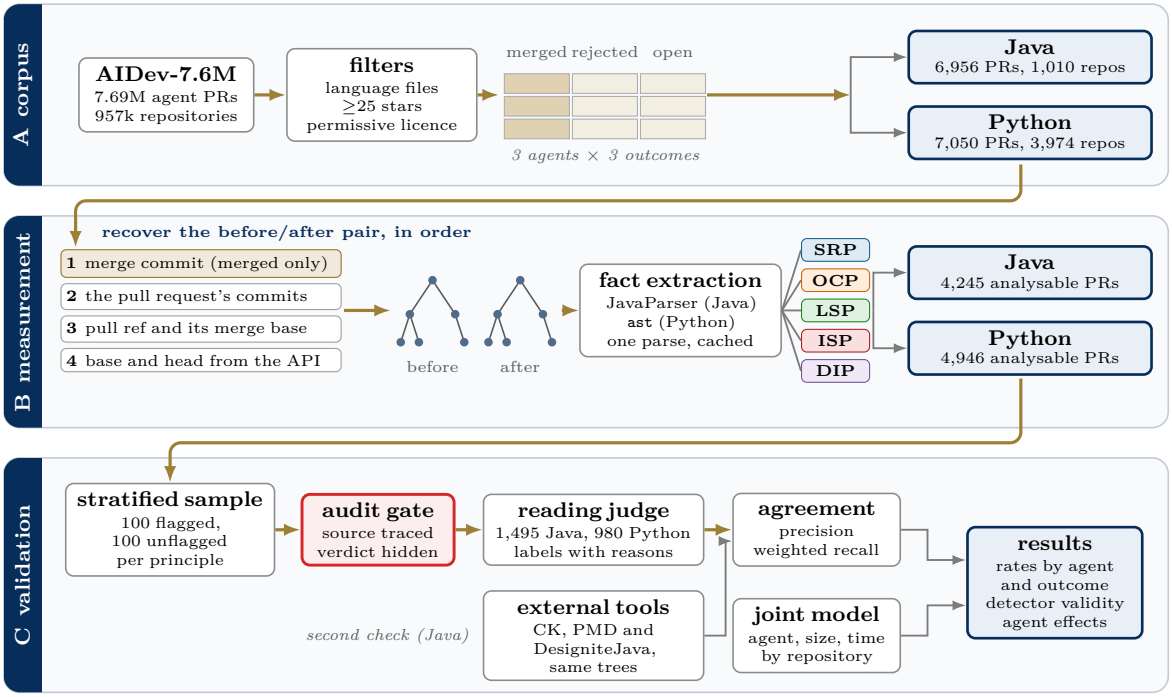


Figure 4.2: The measurement pipeline.

The four delta detectors share one before/after tree pair per pull request, recovered by the four methods of Section 4.2, read with `git archive`, parsed once and cached by content hash. Each pull request is processed independently. Permanent outcomes are kept, temporary failures are retried, and every attempt is recorded, so attrition can be audited.

4.6 Validation Procedure

For each principle a stratified sample is drawn: 100 entities the detector flagged and 100 it did not, after the same production filter the report uses. The sampling seed is recorded. A class, chain, override pair or contract that several pull requests touched is counted once, so that the sample is not drawn towards entities many requests happen to touch. The deduplication keeps the first observation of an entity, while the report keeps its largest observation; the two rules disagree for 123 of the roughly 7,600 flagged Java classes (1.6%), because a class can be flagged in the tree of one pull request and not in another's. The judge reads the source of each entity and writes a binary label (`llm_<principle>_risk`) and a reason; for LSP it sees the overriding method next to its base method. The flag is scored against the label as precision and recall, overall and per agent.

Thresholds for all five detectors were set from corpus percentiles before any label existed. Two refinements were derived later from the false positives of the first sample: restricting the Liskov flag to refused bequest, and requiring interface contracts of at least 30 members. Scoring a refinement on the sample it was derived from overstates its precision. Both were therefore scored again on a holdout: 200 further entities per principle, drawn with a different seed from the same population, excluding every row judged before, processed through the same audit and judged with the same rubric before the rounds were compared. Every earlier round is excluded, not only the most recent one, because a sample that shaped a detector can no longer estimate its performance.

4.6.1 A Second Check: Independent Analysers

A judge that reads the same code as the detector can make the same mistake, for example treating a large class as a class that does too much. For Java, three independently developed tools were therefore run on the same source trees and their verdicts compared with the detectors'.

The comparison uses one tree per repository, the after tree of the repository's most recent pull request, for a seeded random sample of 150 corpus repositories. CK 0.7.0 provides per-class metrics, PMD 7.7.0 design rules and DesigniteJava 1.1.2 design smells. All three run with their default thresholds, because tuning them towards agreement would defeat the purpose.

Three points matter for interpretation. First, agreement is reported symmetrically, with no tool treated as ground truth, and the tools are also compared with each other. Second, CK's `lcom` is LCOM1 and its `lcom*` is the Henderson-Sellers variant (Henderson-Sellers, 1996), while this study uses normalised LCOM4, so cohesion is compared by rank correlation only. Third, each tool fails silently by writing an empty result, which would read as a clean repository. The status of every tool on every repository is recorded, and

failed repositories are excluded from that tool's comparison.

Rank correlation is unaffected by monotone rescaling. A high Spearman correlation on a size metric therefore shows agreement on ordering, not on scale.

4.7 Results

4.7.1 Detector Validation (RQ2)

Table 4.8 gives agreement with the reading judge over all 1,495 judged Java entities. The judge saw the source of each entity without the detector's verdict, and every judged row traces back to the file the judge was shown. Recall is weighted: each stratum is scaled to the size of the population the sample was drawn from, and the 95% interval is a percentile bootstrap resampled within each stratum. The Liskov and Interface Segregation rows include the holdout round.

The Dependency Inversion detector flags 195 pull requests in the whole corpus, few enough to judge all of them. Its precision of 0.892 (174 of 195) is therefore exact for the corpus. The first sample of 100 flagged requests had estimated 0.890 (95% interval 0.814–0.937), so on the one principle where the sampling procedure can be checked against a census, it was close to unbiased. A census of Single Responsibility would require judging 28,756 further entities.

Table 4.8: Java detector agreement with the reading judge.

Principle	Judged	TP	FP	FN	Precision	Recall (weighted)
SRP	200	35	65	1	0.350	0.828 (0.579–1.000)
OCP	200	33	67	2	0.330	0.742 (0.512–1.000)
LSP	400	84	116	3	0.420	0.386 (0.210–1.000)
ISP	400	67	133	9	0.335	0.464 (0.331–0.680)
DIP†	295	174	21	0	0.892	1.000‡

†DIP's flagged stratum is a census of all 195 flagged pull requests. ‡No unflagged entity was judged a violation (0 of 100), so the interval is degenerate and the count is given instead. Pooled over all judged rounds; computed by `labels/score_java_labels.py`.

Figure 4.3 plots these values next to the Python values of Section 4.8.3. Dependency Inversion is precise and complete in both languages, most other detectors have precision below 0.5, and the Java Liskov detector reaches 1.000 when restricted to refused bequest.

Why recall is weighted. Each sample is half flagged and half unflagged, but the population is not. For Liskov it holds 574 flagged override pairs against 25,594 unflagged ones, so one sampled unflagged pair stands for about forty-five times as many entities as one sampled flagged pair. Treating the two strata as equally common would give a Liskov recall of 0.966; weighting them gives 0.386. The intervals are wide because a weighted

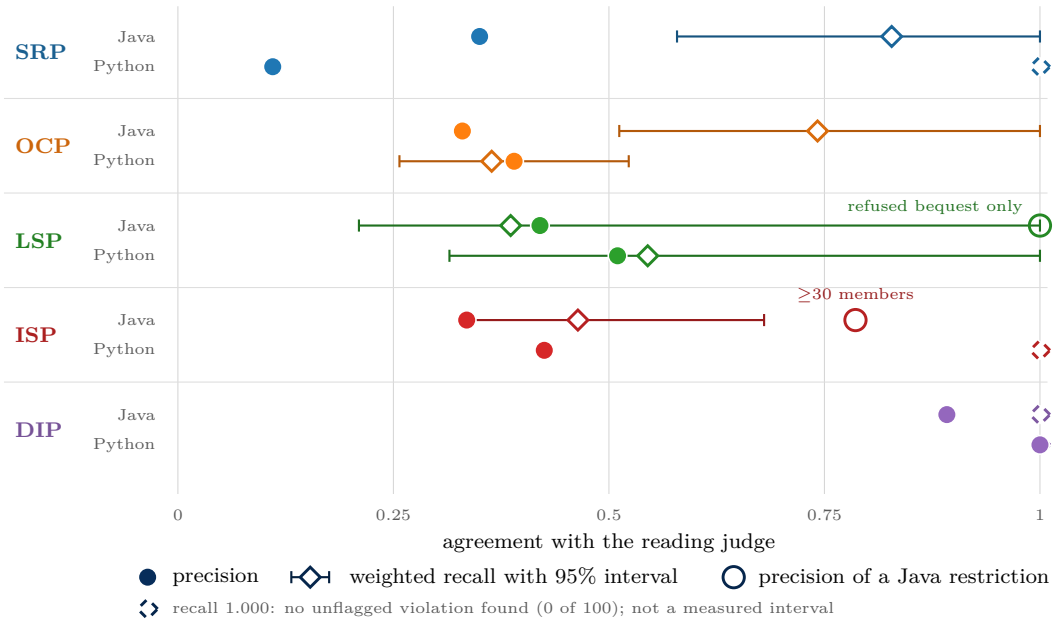


Figure 4.3: Detector precision and weighted recall in Java and Python.

recall depends on the few violations found in the unflagged stratum, three for Liskov in 200 rows, multiplied by a large factor.

The population used for this weighting is the one the sample was drawn from: production entities, counted once each (Section 4.6). Weighting with the raw result rows instead, which include test code and repeat a class once per pull request that touched it, would describe a population the study does not report and would move four of the five figures, Interface Segregation by almost nine points.

For SRP, OCP and DIP the detectors miss few violations and over-report many, so their flag rates are upper bounds on what the judge would confirm. For LSP and ISP they are not: these detectors find the violations with a clear structural form and miss the rest. The three Liskov violations found in the unflagged stratum show what is missed. A Vaadin root node throws `IllegalStateException` for any non-null parent where its base accepts any parent; a CommCare screen controller overrides a method that disables a login button with an empty body; and an nd4j operation implements `setY` as `this.y = null`, discarding its argument. None of them throws where the detector looks, and one does nothing at all.

Precision is below 0.5 for four of the five detectors. A structural detector answers a question about the shape of code, while the principles concern meaning, and these figures measure that gap.

Precision by agent. If a detector over-flags one agent more than another, it creates an agent difference that is not in the code. Table 4.9 gives precision by agent, as true positives over flags judged.

The OpenAI Codex column rests on 7 to 26 flags per principle, so its values are

Table 4.9: Java detector precision by agent.

Principle	Claude Code	GitHub Copilot	OpenAI Codex
SRP	0.327 (18/55)	0.263 (10/38)	1.000 (7/7)
OCP	0.338 (23/68)	0.280 (7/25)	0.429 (3/7)
LSP	0.500 (53/106)	0.324 (22/68)	0.346 (9/26)
ISP	0.422 (35/83)	0.280 (26/93)	0.250 (6/24)
DIP	0.792 (57/72)	0.959 (94/98)	0.920 (23/25)

unstable; its 1.000 for Single Responsibility is 7 of 7. Claude Code and Copilot have 25 to 106 flags each. On four of the five principles Claude Code's precision is the higher of the two, so the detectors are not more likely to over-flag Claude Code, and its higher flag rates in Section 4.7.3 do not come from the detectors. Dependency Inversion is the exception: 0.792 for Claude Code against 0.959 for Copilot over the census, so part of Claude Code's higher DIP rate may be over-flagging. Codex's flag rates are computed over all its 770 pull requests; only its per-agent precision is uncertain.

The judge is a Claude model, and Claude Code is one of the agents evaluated. A judge that read Claude Code's output more leniently would confirm fewer of its flags, since leniency means seeing fewer violations. On four principles Claude Code's flags are confirmed more often, not less. This test is not conclusive: the cells are small, and it cannot detect a bias that affects the judge and the detector in the same way.

Two refinements. Two detectors became much more precise after a restriction suggested by the labels.

The Liskov flag originally combined three signals, and their precision differs sharply. Refused bequest, where a base method with a working body is overridden by a method that only throws `UnsupportedOperationException`, was correct on 31 of 31 flags. Weakened postcondition was correct on 6 of 36 and strengthened precondition on 3 of 33. Restricted to refused bequest, the flag has precision 1.000 (95% interval 0.943–1.000) on 64 flags over both rounds.

The Interface Segregation detector becomes more precise as contract width grows, and the change is abrupt, as Table 4.10 shows for the sample from which the width condition was derived. Requiring at least 30 members raises precision from 0.320 to 0.862 (0.694–0.945) on that sample.

The holdout. Each refinement was then scored on 200 new entities, drawn with a different seed and excluding every previously judged row. Table 4.11 compares the result with the sample from which the refinement was derived.

The Liskov restriction held: no false positive in 64 flags across two samples. The width condition lost 16 points, and its derivation value of 0.862 lies outside the holdout's

Table 4.10: ISP precision by contract width.

Members	Flags judged	True	Precision
0–17	43	5	0.116
18–29	28	2	0.071
30–49	14	11	0.786
50–99	10	9	0.900
≥100	5	5	1.000

Table 4.11: The two refinements on the derivation sample and on a holdout.

Refinement	Derivation sample	Holdout	Pooled
LSP, refused bequest only	1.000 (31/31)	1.000 (33/33)	1.000 (64/64)
ISP, ≥30 members	0.862 (25/29)	0.704 (19/27)	0.786 (44/56)

own interval of 0.515 to 0.841. The two rules have different forms. Refused bequest is categorical: a method body either is only a throw or it is not, and nothing can be tuned. The width of 30 is a number chosen where the false positives stopped, and a number chosen that way absorbs part of the noise of the sample. Both refinements improve precision, and the pooled figures are the ones to quote, but only the categorical rule kept its derivation value.

Judge consistency. 174 entities were judged twice, mainly because the Dependency Inversion detector flags only 195 pull requests, so a later sample of 100 repeats earlier rows. The earlier labels were not consulted during the later round. The two readings agree on 89.7% of the entities (156 of 174), from 84.0% for ISP to 92.2% for DIP. The earlier reading is the first labelling round, which was later discarded because its provenance could not be audited (Appendix A); its labels are used here only to measure how stable the judge's verdicts are, and nowhere else in this thesis. The figure describes the judge, not the detectors, and limits the accuracy with which any agreement figure in Table 4.8 can be known.

4.7.2 Agreement with Independent Analysers

CK 0.7.0, PMD 7.7.0 and DesigniteJava 1.1.2 were run with default thresholds on the head tree of 150 randomly sampled repositories, giving 111,029 production types in common with this study's analysis.

Size and complexity agree; cohesion does not. Rank correlation with CK is high for size and complexity and weak for cohesion, as Table 4.12 shows for the 101,153 production types analysed by both tools, with CK's internal agreement as a control. CK's own

two cohesion variants correlate at only 0.490 with each other, and one of them at 0.240 with a third CK cohesion measure. This study's LCOM agrees with CK's LCOM1 about as well as CK's cohesion measures agree with each other. Cohesion measures disagree in general, which is one reason cohesion is only one of six hits in the rubric.

Table 4.12: Rank correlation with CK.

Comparison	Spearman ρ	Reading
ours ~ CK weighted methods	0.977	strong
ours ~ CK method count	0.969	strong
ours ~ CK lines of code	0.944	strong
ours (largest method complexity) ~ CK response for class	0.789	moderate
ours ~ CK `lcom` (LCOM1)	0.288	weak
ours ~ CK `lcom*`	0.203	weak
control: CK against itself		
CK `lcom` ~ CK `lcom*`	0.490	
CK `lcom` ~ CK `tcc`	0.388	
CK `lcom*` ~ CK `tcc`	0.240	

81.1% of the classes flagged by the SRP detector are in CK's top decile by weighted methods, and 79.7% in its top decile by lines of code.

Three God Class detectors compared. PMD's `GodClass` rule and DesigniteJava's *Insufficient Modularization* and *Multifaceted Abstraction* smells target the same problem as the SRP detector, so the three can be compared on the 82,553 production types that all three analysed (Table 4.13). In each pair, precision and recall treat the second tool as the reference; no tool is ground truth.

Table 4.13: Agreement between three God Class detectors.

Pair	Cohen's κ	Precision	Recall	Flag rate
ours ~ PMD `GodClass`	0.574	0.435	0.940	ours 7.7%
ours ~ DesigniteJava	0.531	0.494	0.651	PMD 3.5%
PMD ~ DesigniteJava	0.384	0.543	0.331	Designite 5.8%

PMD and DesigniteJava agree with each other at $\kappa = 0.384$, less than either agrees with this study's rubric. Of the rubric's 6,326 flags, 4,304 (68.0%) are also flagged by at least one of the two tools, and 1,573 by both. Against PMD's broader rule of any two design rules firing together, agreement reaches $\kappa = 0.701$. This does not show that the rubric is correct. It shows that its disagreement with published detectors is no larger than their disagreement with each other.

Two divergences explained by the unit of measurement. Agreement on substitutability is zero ($\kappa = 0.000$). DesigniteJava's *Rebellious Hierarchy* is reported on the supertype whose contract is broken, while this study's detector flags the subclass that refuses the inherited method, so the two name different classes for the same fault.

For Dependency Inversion, the two tools agree at $\kappa = 0.389$ when compared by compilation unit. Aggregated to packages, DesigniteJava's unit, agreement falls to $\kappa = 0.080$ while recall rises to 0.924: the detector finds nearly every cyclic package DesigniteJava reports but assigns it to smaller units. Interface Segregation has no counterpart in the three tools and is validated by the reading judge alone.

Open/Closed has no external corroboration. DesigniteJava's *Missing Hierarchy* and *Unexploited Encapsulation* both describe conditional logic on type codes, which is what the dispatch-chain detector looks for. Both tools produced output for 138 cross-check repositories, and over the 63,326 production types they contain the two detectors agree at $\kappa = 0.122$: the detector flags 886 types (1.40%), DesigniteJava 299 (0.47%), and 76 types are flagged by both. Of DesigniteJava's flags, 25.4% are also flagged here; of this study's flags, 8.6% are also flagged by DesigniteJava. Agreement is therefore an order of magnitude weaker than for God Class, and the Open/Closed flag rate is three times DesigniteJava's, which alone caps κ. Two differences in definition account for most of the gap. DesigniteJava requires that a hierarchy be available or warranted before it reports the smell, whereas the detector fires on chain width alone once the subject is not an enumeration; and DesigniteJava reports at the level of the abstraction that should have existed, not the method body holding the chain. The reading judge, which does read the body, scored this detector at 0.330 precision, close to the other three sampled principles. The Open/Closed proxy therefore measures something narrower than the published smell, and it rests on the judge alone.

Coverage. DesigniteJava produced results for 141 of the 150 repositories, 92.1% of this study's types joined to a CK row, and PMD could not analyse 252 files. Types affected by a tool failure are excluded from that tool's comparison.

4.7.3 Agent Differentiation (RQ1)

Table 4.14 gives the flag rate of each principle by agent, pooled, with the macro average over repositories in brackets. Where the two disagree, the macro average is preferred. SRP is measured over classes the agent authored; the other four principles over pull requests. The ordering depends on change size (Table 4.15) and is adjusted in Table 4.16.

Claude Code has the highest pooled rate on all five principles and Codex the lowest on four. Three qualifications apply.

Table 4.14: Java flag rate by agent.

Principle	Claude Code	GitHub Copilot	OpenAI Codex
SRP (authored classes)	9.5% (8.4%)	5.1% (3.5%)	6.8% (7.4%)
OCP	10.6% (10.1%)	6.4% (3.0%)	2.1% (2.2%)
ISP	8.7% (6.0%)	6.7% (4.0%)	2.9% (5.0%)
DIP	6.5% (5.1%)	4.1% (2.8%)	3.2% (5.4%)
LSP	0.7% (0.3%)	0.3% (0.1%)	0.0% (0.0%)

First, the agents worked in different periods. The median pull request dates are August 2025 for Codex, January 2026 for Copilot and March 2026 for Claude Code, so each agent's tasks come from a different mix of repositories and framework versions. The joint model below includes time.

Second, the pooled and macro rates disagree for Codex on ISP and DIP: its macro rate is above Copilot's while its pooled rate is below. Codex's flagged pull requests are spread over many repositories while most of its unflagged ones come from a few, and the macro figure is the one to use.

Third, Codex has 770 analysable pull requests against 2,366 for Copilot, so its intervals are wider.

Change size. Cynthia et al. (2026) find that differences between agents in raw issue counts largely disappear after normalising by code churn. The same confound applies here. The agents write changes of different sizes: the mean number of Java files touched is 46.4 for Claude Code, 18.5 for Copilot and 10.9 for OpenAI Codex, and 63.0% of Codex's pull requests touch three files or fewer, against about 49% for the other two. Flag rates rise with size for every agent and principle, as expected for delta detectors, which have more chances to fire when more code changes.

Table 4.15: Java flag rate by agent and change size.

Principle	Files touched	Claude Code	Copilot	OpenAI Codex
OCP	1–3	1.3%	1.9%	0.4%
	4–10	7.0%	3.5%	1.9%
	11+	31.3%	19.3%	8.9%
ISP	1–3	0.4%	0.9%	0.6%
	4–10	7.7%	6.0%	4.3%
	11+	24.7%	19.6%	9.7%
DIP	1–3	0.4%	0.5%	0.8%
	4–10	1.8%	2.4%	3.7%
	11+	22.3%	13.7%	12.1%

As Table 4.15 shows, within bands of change size (Java files touched) the pooled

ordering holds only for the largest changes. Among pull requests touching 11 or more files, Claude Code flags 31.3% on Open/Closed against 8.9% for Codex. Among those touching three files or fewer, Copilot has the highest rate on Open/Closed and Interface Segregation, and Codex has the highest rate on Dependency Inversion in the two smaller bands. Part of the pooled difference is therefore a difference in how much the agents change at once.

Joint model. Size bands, macro averages and median dates each control one confound at a time, but the confounds are related: an agent that writes larger changes may also work in different repositories and in a different period. For each principle, a logistic regression of the pull request flag on agent, change size and time was fitted over all 4,245 analysable pull requests in 790 repositories:

$$\operatorname{logit} \Pr(\text{flag}) \;=\; \beta_0 + \beta_{\text{agent}} + \beta_1 \log(1 + \text{files touched}) + \beta_2 \,\text{months}.$$

It was estimated with generalised estimating equations, an exchangeable working correlation within repository and cluster-robust standard errors, so that pull requests from the same repository are not treated as independent. Copilot, the largest group, is the reference agent. Table 4.16 reports the adjusted odds ratios with 95% confidence intervals. An agent coefficient compares the agent with Copilot at the same change size and time, with repository clustering. Liskov is not fitted: 16 flagged pull requests cannot support the model. Single Responsibility has no pull-request flag in Java (it is scored per class) and is compared through the authored-versus-edited contrast.

Table 4.16: Adjusted odds ratios from the joint model for Java.

Term		**OCP**		**ISP**		**DIP**	
Claude Code	vs Copilot	2.22	(1.21–4.07)	1.11	(0.63–1.95)	1.29	(0.50–3.36)
OpenAI Codex	vs Copilot	0.57	(0.24–1.37)	0.72	(0.40–1.32)	1.68	(0.79–3.56)
Change size	per doubling	1.89	(1.75–2.04)	1.88	(1.73–2.04)	1.95	(1.77–2.16)
Time	per month	1.00	(0.91–1.10)	1.02	(0.95–1.10)	1.03	(0.93–1.15)
flagged / total		286 / 4,245		276 / 4,245		195 / 4,245	

Change size is the strongest predictor. Each doubling of the number of Java files a pull request touches multiplies the odds of a flag by about 1.9 on all three principles ($p < 10^{-4}$). No agent coefficient is as large or as certain.

One agent difference remains after adjustment. Claude Code's odds of an Open/Closed flag are 2.22 times Copilot's at the same size and time (1.21–4.07, $p = 0.010$). For Interface Segregation and Dependency Inversion the odds ratios fall to 1.11 and 1.29 with intervals that include 1, so most of the ordering in Table 4.14 is explained by change size. No Codex coefficient is significant.

The model tests six agent coefficients. With a Bonferroni correction the threshold is $0.05/6 = 0.0083$, and the Open/Closed coefficient ($p = 0.010$) does not pass it on its own. The effect is claimed because it replicates in the independent Python corpus, where $p = 0.0001$ (Section 4.8.4).

Time has no detectable effect. The monthly coefficient is close to 1 on every principle (1.00 to 1.03, all $p > 0.5$). Once size and repository are accounted for, when a pull request was written does not predict whether it is flagged in this corpus and period.

The model does not make the design experimental. Agents chose their own tasks in repositories this study does not control, so the coefficients are adjusted associations, not causal effects. A difference in the kind of task each agent received would still appear in its coefficient. Adjusting for size also removes any part of an agent difference that operates through the size of the changes the agent chooses to make.

Codex introduced no refused-bequest violation in 770 pull requests, and only 16 of the 4,245 pull requests introduced one. The detector does find such violations in the code these pull requests touched (Section 4.7.4). The rate is low because a new violation requires writing a base method and an override that contradict each other.

Authored and edited code. The Single Responsibility detector works per class, so it can compare classes an agent created with existing classes it edited (Table 4.17).

Table 4.17: Java SRP flag rate for authored and edited classes.

	Authored	**Edited**	**All touched**
Pooled	7.7%	13.5%	12.2%
Macro over repositories	5.6%	20.9%	18.2%
Classes	13,712	48,433	62,145

Classes the agents wrote are flagged at about half the rate of the existing classes they edited, and the gap is larger when averaged over repositories. The two groups differ in more than authorship: new classes are smaller and younger, and classes gain responsibilities over years of maintenance. The comparison therefore does not show that agents write better classes than the developers before them. It does show that most flagged classes in this corpus were not written by the agents.

Figure 4.4 shows the same contrast for each agent, in Java and in Python, for production code and averaged over repositories. In every case the edited classes are flagged more often, and the agents are close to each other on edited classes.

Merged and rejected code. The outcome comparison uses the tier-matched subset (methods 2, 3 and 4), where merged and unmerged requests recover their before tree in the same way. Of the 1,177 unmerged pull requests in this subset, only 171 were ever

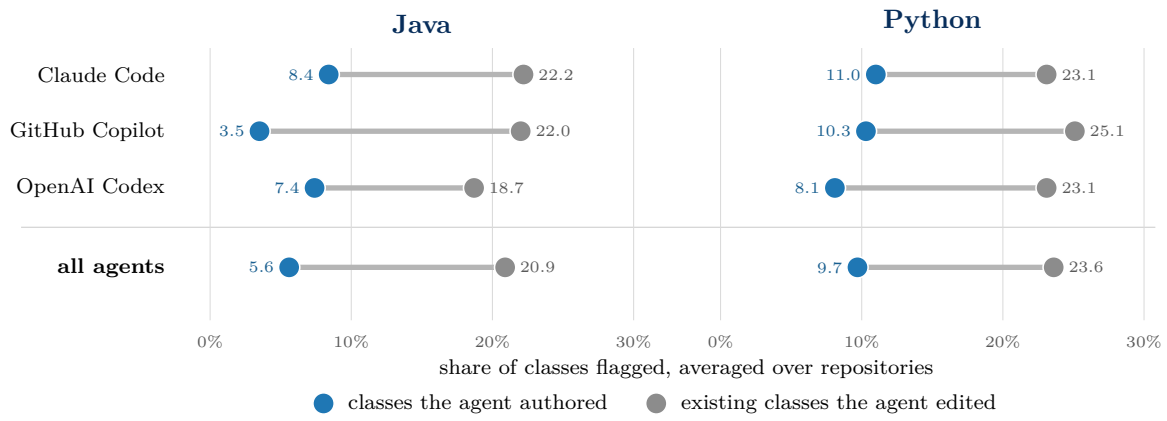


Figure 4.4: SRP flag rate for authored and edited classes.

reviewed. Table 4.18 gives the flag rate by outcome stratum; only the `changes_requested` and `reviewed_then_closed` rows record a maintainer's judgement of the code.

Table 4.18: Java flag rate by outcome stratum.

Stratum	n	OCP	ISP	DIP	LSP
merged	1,521	8.3%	7.5%	4.6%	0.5%
`changes_requested`	36	5.6%	16.7%	13.9%	0.0%
`reviewed_then_closed`	135	12.6%	12.6%	12.6%	0.7%
`closed_unreviewed`	1,006	7.5%	5.5%	5.6%	0.5%
open	490	8.8%	9.2%	5.9%	0.2%

The table supports one firm result and one tentative one.

Merged code is indistinguishable from unreviewed closed code. The 1,006 `closed_unreviewed` requests make up most of the unmerged set, and their rates are within two points of the merged rates on every principle. This comparison has enough requests to be informative: whether a pull request was superseded or abandoned is unrelated to its measured structure.

Reviewed and rejected code has higher rates. Both reviewed strata exceed merged on Interface Segregation (16.7% and 12.6% against 7.5%) and Dependency Inversion (13.9% and 12.6% against 4.6%). For `reviewed_then_closed` the Wilson interval on Dependency Inversion, 8.0–19.2%, does not overlap the merged interval of 3.7–5.8%. These strata are small, however (36 and 135 requests), and 6 of the 17 flagged `reviewed_then_closed` requests come from one repository.

For these signals, then, a corpus of merged code is not a biased sample relative to code nobody reviewed. Whether review itself selects on these properties cannot be answered with 171 reviewed rejections; Section 5.7 proposes a study designed for it.

4.7.4 Locus of Risk (RQ1)

For the relation-level principles a violation is charged to a pull request when either end of the relation is in a touched file. Touched is not authored: a request that edits one line of a subclass is charged with the override pair it belongs to. Only SRP separates authored from edited code. The comparison below therefore gives an upper bound on how much of the relation-level risk the agents could be responsible for.

A violation present in the code a pull request touched may have been written by anyone at any time. A violation the pull request introduced is one the before/after comparison attributes to this change. Table 4.19 compares the two, with the flagged counts corrected by the precision in Table 4.8. Present counts are distinct production entities. For LSP both columns use the refused-bequest flag (precision 1.000); for DIP the detector only records new cycles, so both columns count the same pull requests.

Table 4.19: Java violations present and introduced.

Principle	Flagged present	Est. genuine	PRs that introduced one
SRP	7,501 classes	≈2,600	n/a
OCP	2,343 chains	≈770	286 (2.2% corrected)
ISP	816 contracts	≈270	276 (2.2% corrected)
LSP	183 override pairs	183	16 (0.4% corrected)
DIP	195 pull requests	174	195 (4.1% corrected)

Figure 4.5 plots the same comparison for Java and Python on a log scale: flagged violations present in the production code the agents touched (grey) against pull requests that introduced one (colour). The Liskov row uses refused bequest in both languages, and because the Dependency Inversion detector records only new cycles, its two counts coincide.

After precision correction, between 0.4% and 4.1% of pull requests introduce a violation, depending on the principle, in code that already contains hundreds to thousands of flagged entities. For Liskov Substitution the touched code contains 183 refused-bequest relationships outside test and generated code, and 16 pull requests introduced one.

The Liskov detector has the lowest recall of the five (0.386, 0.210–1.000, for the three-signal flag), and it undercounts in the 718 pull requests with less than 50% symbol resolution. Both effects mean that more violations are present than counted, which would widen the gap between present and introduced violations, not close it.

The explanation is structural. A Single Responsibility or Open/Closed violation lies inside one class or one conditional, which a single pull request can write. A Liskov, Interface Segregation or Dependency Inversion violation lies in a relation: a base and its override, an interface and its implementers, a cycle among modules. One contribution rarely writes both ends, so these violations build up over many changes and authors.

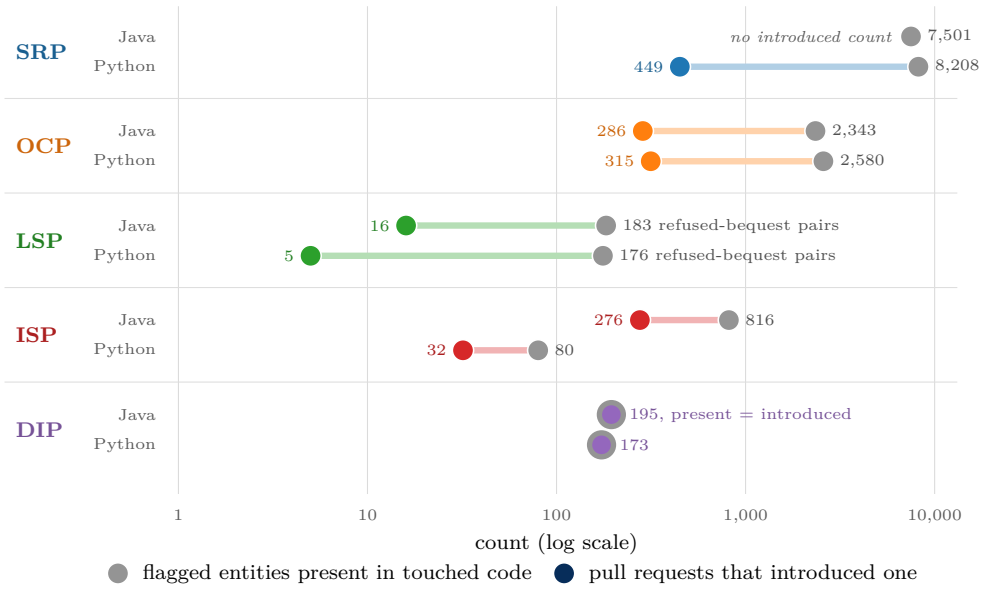


Figure 4.5: Violations present and introduced, in Java and Python.

A consequence for practice is that review of one change at a time can catch problems inside a unit but will rarely see problems between units. These require a view of the repository over time.

4.7.5 Cross-Cutting Findings

Finding 1: the type system limits which Liskov violations can be written. Three of the signals used to detect Liskov violations in a dynamically typed language cannot occur in Java code that compiles: a changed parameter list makes an overload, parameter types are invariant, and visibility cannot be reduced. The Java detector therefore uses only signals the compiler does not check. Of those, refused bequest is correct on 31 of 31 flags, while the two behavioural signals are correct on 6 of 36 and 3 of 33. The Python results test the other side of this result (Section 4.8.3).

Finding 2: width predicts Interface Segregation violations but not Open/Closed violations. Contract width separates Interface Segregation flags sharply (Table 4.10): precision is 0.071 for 18 to 29 members and 0.900 for 50 to 99. The number of branches in a dispatch chain does not separate Open/Closed flags: as Table 4.20 shows for the validation sample, precision stays between 0.270 and 0.400 in every band.

The difference follows from the principles. Interface Segregation is violated by breadth, so width measures it fairly directly. Open/Closed concerns whether a set of variants can grow. A 64-branch switch over PostgreSQL type identifiers is closed, because a wire protocol fixes the set; a five-branch switch over supported storage backends is open, because the next release may add one. Counting branches cannot tell them apart.

Table 4.20: OCP precision by dispatch chain length.

Branches	Flags judged	True	Precision
0–5	37	10	0.270
6–9	23	9	0.391
10–15	19	6	0.316
16–30	10	4	0.400
≥31	11	4	0.364

Finding 3: the closed sets that matter are external specifications. The Open/Closed suppressor recognises sets fixed by the language, and it removed none of the 100 flagged rows in the validation sample. The false positives show why: the closed sets in practice are fixed by external specifications, such as JDBC type codes, protobuf scalar types, FHIR binding strengths, Unicode blocks, AWT cursor constants and Android key codes, or by generated parser tables, where a 356-branch keyword switch is generated from a grammar file. A suppressor that recognises such specification enumerations is a concrete way to make the detector more precise.

Finding 4: most design problems in the touched code were already there. After precision correction, the Java code these agents touched contains about 2,600 Single Responsibility violations and 183 refused-bequest relationships, while between 0.4% and 4.1% of pull requests introduced a violation of a given principle. The difference is largest for the relation-level principles, for the reason given in Section 4.7.4.

Finding 5: classes written by agents are flagged less often than the classes they edit. 7.7% against 13.5% pooled, and 5.6% against 20.9% averaged over repositories (Table 4.17). The two groups differ in age and size, so this describes the corpus and does not show that agents write better classes than people. It does contradict the assumption that agent code is the main source of structural problems in these repositories.

Finding 6: merged code is not a biased sample of unreviewed agent code. With the reconstruction method held constant, merged pull requests are structurally indistinguishable from the 1,006 closed without review (Table 4.18). For these signals, merged-only corpora are therefore usable. The two reviewed strata, with 36 and 135 requests, show higher rates on Interface Segregation and Dependency Inversion, so whether reviewers act on structure is still an open question that needs many more reviewed rejections.

4.8 Results in Python (RQ1 and RQ2)

The Java results could describe the agents or only how the agents write Java. Change size dominates every principle, one agent difference remains after adjustment, and most design problems in the touched code were already there. Each of these could depend on a language with compulsory classes, declared contracts and a compiler that rejects a family of substitutability faults. This section applies the same framework, agents, curation rules, validation procedure and statistical model to Python.

Python differs from Java on each of these points. A module need not declare a class; a contract can be an `abc.ABC`, a `typing.Protocol` or a base class whose methods raise `NotImplementedError`; nothing checks subtyping; and the faults `javac` rejects are legal until the program runs.

4.8.1 Corpus

The Python corpus comes from the same AIDev release with the same repository filters (at least 25 stars and a licence from the allowlist of Section 4.2), keeping pull requests that change Python files. Two sampling choices differ from Java. The draw is balanced exactly on agent, with 1,400 merged, 700 rejected and 250 open requests per agent (2,350 per agent). It is also stratified by repository, so no repository contributes more than 0.4% of any agent and outcome cell, against 17.2% for the most concentrated Java cell. The curated corpus contains 7,050 pull requests from 3,974 repositories, created between February 2025 and March 2026.

Table 4.21: Analysable Python pull requests by agent and outcome.

Agent	Merged	Rejected	Open	Total
Claude Code	940	540	182	1,662
GitHub Copilot	885	407	178	1,470
OpenAI Codex	1,069	549	196	1,814
Total	2,894	1,496	556	4,946

Reconstruction uses the same four methods as for Java. Of the 7,050 curated requests, 4,946 from 2,921 repositories are analysable (Table 4.22), and Table 4.21 breaks them down by agent and outcome. The three agents are more evenly represented than in Java, and OpenAI Codex, the smallest Java group, is the largest Python group.

Some of this attrition is source loss, not a scoping decision. 496 Python pull requests (7.0%) could not be retrieved: the dataset has no file list for them, they have no merge commit or recorded commits, and their branch has been deleted from GitHub. The loss is uneven: 5.1% of merged requests against 11.0% of rejected ones, and 14.3% of Copilot's requests against 3.6% and 3.1% for the other agents. The lost requests were not replaced,

Table 4.22: From the curated Python corpus to the analysable corpus.

Status	PRs	Meaning
`ok`	4,946	analysable: valid before/after pair, Python changed
`no_touched_files`	1,816	no Python file left to analyse
`no_touched_modules`	232	files changed, none resolvable to a module
`no_delta_pair`	40	no method recovered a distinct before tree
`repo_too_large`	14	source tree above the parsing limit
`archive_failed`	2	a tree object missing, or a 4.9 GB data tree refused
Total	7,050	

because replacements could only come from requests whose branches still exist, which would tie retrievability to the outcome being compared.

4.8.2 Changes to the Instrument

Only the fact extractor was replaced. Python source is parsed with the standard library's `ast` module, without a JVM or a symbol solver, into the same record of types, members, supertypes, imports and member accesses that the Java extractor writes. The detector logic, the delta comparison, the attribution rules, the validation procedure, the report and the joint model are the same code as for Java. Three things changed.

Thresholds were recalibrated before any label existed. Java percentiles do not transfer: Python classes are smaller and Python expresses dispatch differently. A calibration pass over all 7,050 requests recorded every metric with flags switched off, thresholds were set near the 90th percentile, and the result files were deleted and the pass run again, so no flag predates its threshold. Table 4.23 compares the two sets, each calibrated on its own corpus before labelling.

The Liskov detector keeps the signals Java does not need. In Java a changed parameter list makes an overload and parameter types are invariant, so the Java detector does not look for either (Section 4.7.5, Finding 1). In Python both are real faults. Besides refused bequest and a widened return type, the Python detector flags a removed parameter, an added required parameter and a narrowed parameter type.

Contracts are recognised by convention. Python has no `interface` keyword. The detector treats a subclass of `abc.ABC` with abstract members and a `typing.Protocol` as explicit contracts. A base class whose methods only raise `NotImplementedError` is recorded as an implicit contract but not flagged, because in Python this is usually a

Table 4.23: Detector thresholds in Java and Python.

Principle	Java	Python
SRP	six-metric rubric, ≥2 hits; class length 362, WMC 45, 20 methods, method complexity 11, 5 parameters, LCOM 0.86	same rubric, ≥2 hits; class length 250, WMC 36, 10 methods, method complexity 12, 7 parameters, LCOM 0.8
OCP	type chains ≥3 branches, tag chains ≥8	type chains ≥3 branches, value chains ≥5
LSP	refused bequest, strengthened precondition, weakened postcondition (flag: refused bequest)	refused bequest, widened return type, and three signature faults a compiler would reject
ISP	new or widened contract of ≥13 members, or a forced stub	at least two of: more than 7 abstract members, mean implementer coverage below 0.75, an implementer with forced stubs
DIP	new cyclic module pairs through a touched file	new cyclic module pairs through a touched module

template method with optional hooks. The Python flag also needs two signals, so width alone never flags. The Java and Python ISP rates therefore measure different things and are not compared directly.

4.8.3 Detector Validation

The validation procedure is the one used for Java: 100 flagged and 100 unflagged entities per principle, labelled by the same kind of reading judge against a written rubric, with a reason for every label, every label traced to the file the judge was shown, the detector's verdict hidden, and a mechanical audit before judging. Interface Segregation flags only 80 contracts in the Python corpus, and all 80 flagged contracts were judged, so its precision is exact. For Dependency Inversion, 100 of the 173 flagged pull requests were judged. Table 4.24 gives the Python agreement next to the Java figures of Table 4.8 and covers 980 judged entities. Recall is weighted by the size of the population sampled, with a 95% percentile bootstrap resampled within each stratum.

Four of the five detectors are at least as precise in Python as in Java. Single Responsibility is the exception.

Single Responsibility precision falls to a third. The rubric is the same and it still finds large classes. What differs is what a large Python class usually is. Most of the 89 false positives are classes whose size comes from outside the project: a `transformers` model

Table 4.24: Python detector agreement with the reading judge.

	Python					Java
Principle	**Judged**	**TP**	**FP**	**FN**	**Precision / recall**	**Precision / recall**
SRP	200	11	89	0	0.110 / 1.000‡	0.350 / 0.828
OCP	200	39	61	13	0.390 / 0.364 (0.257–0.523)	0.330 / 0.742
LSP	200	51	49	2	0.510 / 0.545 (0.315–1.000)	0.420 / 0.386
ISP†	180	34	46	0	0.425 / 1.000‡	0.335 / 0.464
DIP	200	100	0	0	1.000 / 1.000‡	0.892 / 1.000

†ISP's flagged stratum is a census of all 80 flagged contracts. ‡No unflagged entity was judged a violation (0 of 100), so the interval is degenerate and the count is given instead.

head, a Home Assistant entity, a Lightning callback, a command class whose methods are the subcommands of one tool. Others are clients for a single service or device, or configuration classes. In Python, frameworks set the shape of many classes, so size says less about responsibility.

Liskov signals that a compiler would reject. Of the 51 true positives, 46 of the 51 rest on a signal that `javac` would reject: a removed parameter, an added required parameter or a narrowed parameter type. Only five rest on refused bequest alone. Where no compiler removes these faults, they make up most of the confirmed violations. This supports Finding 1: the Java detector's three signals reflect what Java had already excluded.

Dependency Inversion. The only systematic false positive in the Java census was a cycle charged to a module declared as touched whose text had not changed. The Python judging rule checks this directly by comparing each charged module's text in the two trees, and all 100 judged Python flags passed: each closes a new import cycle through changed code, and no judged unflagged request does. The most common form is a new or edited module that imports its own package root while the root imports it back. The validation shows that the cycles exist, not that they are harmful, so the Python cycle results below mean "introduced an import cycle".

Precision by agent. Table 4.25 gives precision by agent, as true positives over flags judged. The Open/Closed flags, which carry the agent difference that survives adjustment, are confirmed most often for Claude Code, so that difference does not come from noisier flags. The Liskov column is uneven for another reason: two Claude Code pull requests contribute 50 of the 100 flagged rows, one with 14 true positives in 30 and one with none in 20. Without those two requests, Liskov precision is 0.740.

Table 4.25: Python precision by agent.

Principle	Claude Code	GitHub Copilot	OpenAI Codex
SRP	0.130 (6/46)	0.077 (2/26)	0.107 (3/28)
OCP	0.444 (20/45)	0.368 (7/19)	0.333 (12/36)
LSP	0.397 (29/73)	0.733 (11/15)	0.917 (11/12)
ISP	0.479 (23/48)	0.353 (6/17)	0.333 (5/15)
DIP	1.000 (53/53)	1.000 (16/16)	1.000 (31/31)

4.8.4 Agent Differences under the Joint Model

Table 4.26 gives the raw flag rates, pooled, with the macro average over repositories in brackets. SRP is measured over classes the agent authored, the other four principles over pull requests, and the adjusted comparisons are in Table 4.27. As in Java, Claude Code has the highest rate on most principles, and as in Java the agents write changes of different sizes: on average 16.5 Python files per pull request for Claude Code, 7.7 for Copilot and 9.6 for Codex.

Table 4.26: Python flag rate by agent.

Principle	Claude Code	GitHub Copilot	OpenAI Codex
SRP (authored classes)	9.6% (11.0%)	6.6% (10.3%)	8.9% (8.1%)
OCP	9.7% (9.6%)	3.5% (3.1%)	5.7% (6.6%)
ISP	1.3% (1.0%)	0.4% (0.4%)	0.3% (0.1%)
DIP	5.8% (5.6%)	2.0% (1.5%)	2.6% (2.3%)
LSP	1.6% (1.2%)	0.5% (0.3%)	0.8% (0.8%)

The joint model of Section 4.7.3 was fitted without changes over all 4,946 requests in 2,921 repositories, with Copilot as the reference. Python supports all five principles, including a pull-request-level Single Responsibility flag and Liskov with 47 flagged requests. Table 4.27 reports the adjusted odds ratios with 95% confidence intervals. An agent coefficient compares the agent with Copilot at the same change size and time, with repository clustering, and the last rows of the table repeat the Java estimates.

Change size dominates again. Each doubling of the files a pull request touches roughly doubles the odds of a flag, with odds ratios from 2.01 to 2.18 on all five principles ($p < 10^{-4}$), against 1.88 to 1.95 in Java. It is the largest effect in both languages.

The Open/Closed agent difference replicates. Claude Code's odds of an Open/Closed flag are 2.09 (1.44–3.04) times Copilot's at the same size and time, with $p = 0.0001$, against 2.22 (1.21–4.07) in Java. The Python model tests ten agent coefficients, so the Bonferroni threshold is $0.05/10 = 0.005$, and this coefficient passes it. Together with the Java estimate, this is the one agent difference the study supports in both languages.

A second difference is not confirmed. Claude Code's odds of introducing an

Table 4.27: Adjusted odds ratios from the joint model for Python.

Term	SRP	OCP	ISP	DIP	LSP
Claude Code	1.09 (0.93–1.29)	2.09 (1.44–3.04)	1.71 (0.67–4.36)	1.88 (1.17–3.03)	1.81 (0.74–4.42)
OpenAI Codex	1.06 (0.90–1.25)	1.46 (0.98–2.16)	0.53 (0.16–1.80)	1.12 (0.67–1.86)	1.38 (0.54–3.52)
Size, per doubling	2.06 (1.94–2.18)	2.01 (1.88–2.15)	2.17 (1.82–2.58)	2.04 (1.89–2.21)	2.18 (1.89–2.51)
Time, per month	1.05	0.99	0.94	1.06	0.96
flagged / total	2,532 / 4,946	315 / 4,946	32 / 4,946	173 / 4,946	47 / 4,946
Java, Claude Code	n/a	2.22	1.11	1.29	n/a
Java, per doubling	n/a	1.89	1.88	1.95	n/a

import cycle are 1.88 times Copilot's (1.17–3.03, $p = 0.0096$); in Java the ratio was 1.29 with an interval that includes 1. The Python p-value does not pass the Bonferroni threshold, and the Java estimate does not support it, so it is reported as a possible difference that needs confirmation. All other agent coefficients have intervals that include 1.

Time. The monthly coefficient is close to 1 for OCP, ISP, DIP and LSP, as in Java. The pull-request-level Single Responsibility flag, which Java does not have, rises by 5% per month ($p < 10^{-4}$) over a thirteen-month window. Because that flag has precision 0.110, this is a trend in how often large classes are touched or created, not a demonstrated trend in responsibility violations.

4.8.5 Merged and Rejected Code

The outcome comparison follows the Java procedure: tier-matched, and split by stratum. Table 4.28 gives the flag rate by outcome stratum on the tier-matched subset, with flagged counts in brackets; SRP is a per-class measure and is not split this way.

Table 4.28: Python flag rate by outcome stratum.

Stratum	n	OCP	ISP	DIP	LSP
merged	1,734	4.9% (85)	0.4% (7)	3.1% (54)	0.5% (8)
`changes_requested`	33	9.1% (3)	0.0% (0)	9.1% (3)	0.0% (0)
`reviewed_then_closed`	163	7.4% (12)	1.8% (3)	8.6% (14)	1.2% (2)
`closed_unreviewed`	1,300	9.1% (118)	0.9% (12)	4.5% (59)	1.7% (22)
open	553	10.8% (60)	1.1% (6)	5.2% (29)	2.2% (12)

Here the languages differ. In Java, merged code was indistinguishable from code closed without review. In Python, merged pull requests have a lower flag rate than

the unreviewed, reviewed-then-closed and open strata on all four principles; against the 1,300 unreviewed closures the Open/Closed rate is 4.9% against 9.1%. Only the 33 `changes_requested` requests, too few to compare, are lower on two principles.

Part of the gap comes from change size: unmerged requests are larger (median four files against three). With size and agent added to the comparison and repository clustering, unmerged requests still have higher Open/Closed odds (odds ratio 1.57, 1.15–2.15, $p = 0.004$) and higher Liskov odds (2.43, 1.08–5.46, $p = 0.03$). The Interface Segregation (1.71, 0.75–3.91) and Dependency Inversion (1.36, 0.92–2.03) differences are not distinguishable from zero. Open/Closed precision also differs by outcome, 0.239 for merged and 0.500 for rejected requests, and correcting flags by these values widens the gap. Because most unmerged requests were never reviewed, the result means that merged Python code is structurally cleaner than abandoned code, not that reviewers rejected code for its structure.

4.8.6 Locus of Risk

Table 4.29 repeats the comparison of Section 4.7.4, with both columns corrected by the Python precision in Table 4.24, and sets the Python rates next to the Java rates of Table 4.19. Present counts are distinct production entities. For SRP, introduced means a flagged class the pull request newly authored, which the Java table does not report.

Table 4.29: Python violations present and introduced.

	Python			Java
Principle	**Flagged present**	**Est. genuine**	**PRs introduced**	**PRs introduced**
SRP	8,208 classes	≈900	449 (1.0% corrected)	n/a
OCP	2,580 chains	≈1,000	315 (2.5% corrected)	2.2%
ISP	80 contracts	34	32 (0.3% corrected)	2.2%
LSP	2,069 override pairs	≈1,050	47 (0.5% corrected)	0.4%
DIP	173 pull requests	≈173	173 (3.5% corrected)	4.1%

The Python Liskov row counts every Liskov signal. Restricted to refused bequest, as in the Java column and Figure 4.5, the touched code holds 176 relationships and five pull requests introduced one.

The pattern holds. The Python code these agents touched contains 176 refused-bequest relationships outside test and generated code, and five pull requests introduced one; in Java the figures are 183 and 16. Across the five principles, between 0.3% and 3.5% of Python pull requests introduce a genuine violation, in code that already contains tens to thousands of flagged entities.

The comparison of authored and edited classes points the same way. Classes the agents authored are flagged at 8.9% against 12.1% for existing classes they edited, and at 9.7% against 23.6% averaged over repositories. The gap is smaller than in Java (7.7%

against 13.5% pooled, 5.6% against 20.9% averaged), but it has the same direction on both measures.

4.8.7 The Java Refinements Applied to Python

In Java, a categorical restriction (Liskov restricted to refused bequest) held on a holdout, while a numeric one (Interface Segregation restricted to 30 or more members) did not. Both rules were fixed on Java data before any Python label existed, so applying them unchanged to the Python labels is a holdout in a second language. Table 4.30 gives the result; Java counts all contract members, whereas Python counts abstract members, the width its detector records.

Table 4.30: The two Java refinements applied to the Python labels.

Refinement	Java (derivation → holdout)	Python
LSP, refused bequest only	1.000 → 1.000	0.857 (6/7)
ISP, ≥30 members	0.862 → 0.704	0.200 (1/5)

The categorical rule carries over, with six of seven correct; the exception is an override that repeats a refusal its base method already makes for the same case. The numeric rule does not carry over. Both Python counts are small and are not offered as estimates, but they point the same way as the Java holdout: a rule that names a shape of code transferred to a language with a different type system, and a rule that names a quantity did not.

4.8.8 Summary of the Two Languages

As Table 4.31 shows, the Java results can be compared one by one with the Python results.

The results on change size, the Open/Closed difference, pre-existing violations and the fate of fitted rules hold in both languages. The outcome result differs between the languages, and the Single Responsibility screen transfers poorly. Section 5.2 draws the conclusions.

Table 4.31: Java results tested in Python.

Result	**Java**	**Python**
Change size dominates	OR 1.88–1.95 per doubling	holds: OR 2.01–2.18
Open/Closed agent difference	Claude Code OR 2.22 (1.21–4.07)	holds: 2.09 (1.44–3.04), passes Bonferroni
Other agent differences	none	DIP 1.88 (1.17–3.03), not confirmed
Violations present far exceed those introduced	refused bequest 183 present, 16 PRs	holds: 176 present, 5 PRs
Authored classes flagged less than edited	7.7% vs 13.5%	holds: 8.9% vs 12.1%
Categorical rule transfers, numeric rule does not	1.000 vs 0.704 on holdout	holds: 0.857 vs 0.200
Compiler removes a family of LSP faults	inferred from Java's signals	supported: 46 of 51 true positives are compiler-rejectable
Merged code equals unreviewed code	indistinguishable	does not hold: merged lower
Detector precision	0.330–0.892	0.110–1.000; SRP drops to 0.110

Chapter

5

Conclusion

5.1 Summary of the Work

This thesis asked whether the code that autonomous coding agents submit to real projects follows the SOLID design principles. Chapter 1 motivated the question with the distance between benchmark scores and acceptance by maintainers. Chapter 2 showed that existing studies either measure design quality on small sets of tasks or do not compare agents on a common corpus of real contributions. Chapter 3 defined one deterministic detector per principle, validated against an independent reading judge. Chapter 4 applied the design to two corpora of agent pull requests, written by three agents and including merged, rejected and open requests: 4,245 analysable Java pull requests from 790 repositories and 4,946 analysable Python pull requests from 2,921 repositories.

Three design decisions distinguish the study. The corpus is not limited to merged code, so the effect of review can be examined. The validity of each detector is measured: 1,495 Java and 980 Python entities were read and labelled with a reason for each label. And every result was examined in two languages, Java and Python, the second without Java's structural guarantees, and only results found in both languages are called general.

This chapter answers the research questions (Section 5.2), lists the contributions (Section 5.4) and practical implications (Section 5.5), states the limitations (Section 5.6) and proposes future work (Section 5.7).

5.2 Answers to the Research Questions

The two research questions of Section 1.3 are answered in turn. RQ1 is answered by four analyses: differences between agents, differences between outcomes, the locus of risk and the comparison of the two languages. The first three are answered for Java, where they were first established, and the last states which of these answers the Python results support.

The short answer. Agents do violate SOLID, and they do so rarely. After correcting for measured precision, between 0.4% and 4.1% of Java pull requests and between 0.3% and 3.5% of Python pull requests introduce a violation of a given principle. The same code already contains 7,501 flagged classes, 2,343 flagged dispatch chains, 816 flagged contracts and 183 refused-bequest relationships in Java, against 16 pull requests that introduced a refused bequest. The rate at which agents create structural problems is therefore low, and the quantity of structural problems in the code they work in is high. Which agent made the change matters less than how large the change was: each doubling of the files touched multiplies the odds of a flag by 1.88 to 1.95 in Java and 2.01 to 2.18 in Python, and only one agent difference survives that adjustment in both languages.

The detectors agree with the reading judge for two principles and screen for three.

Dependency Inversion reaches precision 0.892 in Java on a census of all 195 flagged pull requests and 1.000 in Python, and the Liskov flag restricted to refused bequest reaches 1.000 on 64 Java flags across a fitted sample and a holdout, and 0.857 in Python. Single Responsibility, Open/Closed and Interface Segregation have precision between 0.110 and 0.425, so their flags locate code worth reading and should not be counted as violations. Against published analysers the Single Responsibility rubric agrees with PMD at $\kappa = 0.574$ and with DesigniteJava at $\kappa = 0.531$, while those two tools agree with each other at $\kappa = 0.384$; the Open/Closed detector is the one that no published tool corroborates, at $\kappa = 0.122$.

RQ1, differences between agents and outcomes. The agents differ less than pooled rates suggest. Claude Code has the highest pooled flag rate on all five principles and OpenAI Codex the lowest on four; Codex introduced no refused-bequest violation in 770 pull requests (Table 4.14).

Much of this ordering reflects change size. Claude Code touches 46.4 Java files per pull request on average and Codex 10.9, and flag rates rise with size for every agent, which matches the churn effect reported by Cynthia et al. (2026). Within size bands (Table 4.15) the ordering holds for the largest changes, where Claude Code flags 31.3% of its Open/Closed changes against 8.9% for Codex, but not for the smallest, where Copilot is highest on two principles and Codex on Dependency Inversion.

The joint model (Table 4.16) accounts for size, time and repository together. Each doubling of the files touched multiplies the odds of a flag by about 1.9 on every modelled principle. One agent difference remains: Claude Code's Open/Closed odds are 2.22 times Copilot's (1.21–4.07). The Interface Segregation and Dependency Inversion differences fall to 1.11 and 1.29, with intervals that include 1. On its own, the Java Open/Closed coefficient does not pass a Bonferroni correction over the six agent coefficients; it is accepted because it replicates in Python. Time has no detectable effect (all $p > 0.5$). The pooled and macro rates disagree for Codex on two principles, and Codex has a third as many pull requests as Copilot, so its estimates are the least precise.

On outcomes, with the reconstruction method held constant, merged pull requests are structurally indistinguishable from those closed without review, which are 1,006 of the 1,177 unmerged requests (Table 4.18). The two strata that record a maintainer's judgement are small (36 and 135 requests) and have higher Interface Segregation and Dependency Inversion rates than merged code, so whether review acts on structure remains open. For these signals, a corpus of merged code is not a biased sample of unreviewed agent code.

RQ1, where the risk lies. Most flagged problems in the touched code were there before the agent's change. After precision correction, the Java code these agents touched

contains about 2,600 Single Responsibility violations and 183 refused-bequest relationships, and between 0.4% and 4.1% of pull requests introduced a violation of a given principle (Table 4.19). For refused bequest, 16 pull requests introduced one.

The reason is structural. Single Responsibility and Open/Closed violations lie inside one unit, which a pull request can write in full. Liskov, Interface Segregation and Dependency Inversion violations lie in relations between units, and one contribution rarely writes both ends. Review of single changes can therefore catch problems inside units but will rarely see problems between units.

RQ1, Java and Python. Most of the Java answers hold in Python. With the same agents, curation rules, validation procedure and model, and thresholds recalibrated on Python before any Python label existed, the Python study reproduces the main Java results (Table 4.31).

Change size dominates in both languages, with odds ratios of 2.01 to 2.18 per doubling in Python against 1.88 to 1.95 in Java. The Open/Closed agent difference replicates: Claude Code's adjusted odds are 2.09 times Copilot's in Python (1.44–3.04, passing the Bonferroni threshold), against 2.22 in Java, and its Python Open/Closed flags are the most often confirmed of the three agents. A possible second difference, more import cycles introduced by Claude Code (1.88, 1.17–3.03), does not pass the correction and is not supported by Java. Pre-existing problems again outnumber new ones: 176 refused-bequest relationships are present against five introducing pull requests, and between 0.3% and 3.5% of Python pull requests introduce a genuine violation. Applied unchanged to the Python labels, the categorical Liskov rule scores six of seven and the numeric width rule one of five. And 46 of the 51 confirmed Python Liskov violations rest on faults that `javac` would reject.

Two results do not transfer. The Single Responsibility screen degrades, from precision 0.350 to 0.110, because the large classes in Python projects are mostly shaped by frameworks. And the outcome result differs: after adjusting for size and agent, merged Python code carries fewer Open/Closed and Liskov flags than unmerged code (odds ratios 1.57 and 2.43 for unmerged requests), while merged and unreviewed Java code were indistinguishable. Since most unmerged requests in both languages were never reviewed, the Python result describes abandoned code, but it means that the Java finding about merged-only corpora does not generalise.

RQ2: can SOLID be measured at scale, and how faithfully? Partly, and the answer differs by principle. Dependency Inversion reaches precision 0.892, measured on every flagged pull request, while the other four detectors have precision between 0.330 and 0.420. Weighted recall is 0.742 to 1.000 for SRP, OCP and DIP, whose flag rates can therefore be read as upper bounds, but 0.386 for Liskov and 0.464 for Interface

Segregation (Table 4.8). In Python, precision is 1.000 for Dependency Inversion, 0.510 for Liskov, 0.425 for Interface Segregation, 0.390 for Open/Closed and 0.110 for Single Responsibility (Table 4.24).

Two detectors improve with a restriction suggested by the labels. Restricting the Liskov flag to refused bequest gives precision 1.000 on 64 flags over two samples. Requiring interface contracts of at least 30 members raises Interface Segregation precision to 0.786 pooled. On a holdout of 200 new entities per principle, the Liskov restriction held (33 of 33) and the width condition fell from 0.862 to 0.704, outside its own holdout interval. The categorical rule kept its precision; the numeric threshold did not.

The remaining gap reflects the difference between structure and intent. Open/Closed precision is flat across dispatch chain lengths, because whether a set of variants can grow does not depend on how many branches it currently has.

The detectors were also compared with three published analysers on 111,029 production types. The Single Responsibility rubric agrees with PMD at $\kappa = 0.574$ and with DesigniteJava at $\kappa = 0.531$, while PMD and DesigniteJava agree at $\kappa = 0.384$ (Table 4.13), and 68.0% of the rubric's flags are also flagged by at least one of the tools. Where the tools diverge, the reason is the unit of measurement: DesigniteJava assigns a substitutability fault to the supertype and cycles to packages, while this study assigns them to the subclass and to compilation units.

5.3 This Study Against the State of the Art

Chapter 2 set out what earlier work reports. As Table 5.1 shows, each of those findings is placed beside the corresponding measurement made here. "Not tested" means the design does not observe the quantity in question, not that the earlier finding is doubted. Two of them are confirmed on real pull requests and two are consistent with the results here. One is not tested by this design, and two address a different question or construct, one of which gives the opposite ordering once the unit of measurement is made explicit.

Two of these entries deserve comment. The comparison with Zhu et al. (2026a) is indirect, since that benchmark uses C++ probes while this study measures Java and Python, and it records no disagreement. That benchmark gives an agent a maintainability task and records whether the agent completes it, so a 4.3% success rate says agents are poor at repairing dependency structure. This study records whether an agent's change creates a violation, and finds that it seldom does. An agent can be both unlikely to fix a cycle and unlikely to create one, and the two results together suggest that agents leave dependency structure roughly as they found it.

The reversal against Pehlivan et al. (2025) follows from what each method can see. A language model reading a short example can recognise a class doing several things, which is why Single Responsibility is easy for it, but it cannot see an import cycle that

Table 5.1: Earlier findings compared with this study.

Earlier finding	This study	Outcome
Code volume predicts structural degradation, the Volume-Quality Inverse Law (Zhu et al., 2026b)	Change size is the dominant term in the joint model, OR 1.88–1.95 per doubling in Java and 2.01–2.18 in Python, on every modelled principle	Confirmed, and it is the strongest result in this thesis
Agent differences disappear once code churn is normalised (Cynthia et al., 2026)	After adjusting for files touched, repository and time, only Claude Code on Open/Closed survives in both languages (2.22 Java, 2.09 Python)	Confirmed, with one exception that replicates
Passing tests does not imply a change a maintainer would accept (Whitfill et al., 2026)	Merged Java pull requests are structurally indistinguishable from those closed without review, and only 171 Java and 196 Python unmerged requests were ever reviewed	Consistent: merge status carries little structural information here
Generated projects reach 91% functional correctness yet carry thousands of design issues (Kashif et al., 2026)	Violations already present in the touched code outnumber those introduced by a factor of about eleven in Java and thirty-five in Python for refused bequest	Consistent, but the unit differs: that study builds new projects, this one measures contributions to existing ones
Agents succeed on 4.3% of dependency-control and 15.2% of responsibility-decomposition tasks, on C++ probes (Zhu et al., 2026a)	Agents seldom introduce dependency or responsibility violations into existing Java and Python repositories	Different question, and a different language: that benchmark asks agents to repair structure, this study asks whether they damage it
Structural erosion grows across 77% of long-horizon trajectories (Orlanski et al., 2026)	The time coefficient is inert for every delta principle in both languages	Not tested: the unit here is one pull request, not a trajectory of successive edits
Single Responsibility is the easiest principle for a language model to detect and Dependency Inversion the hardest (Pehlivan et al., 2025)	For structural detectors the order is reversed: Dependency Inversion is the most precise (0.892 Java, 1.000 Python), Single Responsibility the least (0.350 Java, 0.110 Python)	Different construct: that work measures a model reading examples, this one measures a static rule over a corpus

spans files it was not shown. A static analyser over a whole repository has the opposite profile: the import graph is exactly what it computes, while responsibility is a judgement about intent that no metric captures, which is why the Single Responsibility rubric reaches precision 0.350 in Java and 0.110 in Python. The two findings are compatible and, taken together, argue for combining a graph-based detector with a reading model, not for choosing between them.

5.4 Contributions

C1. A repository-scale measurement instrument for the five SOLID principles, with each detector's precision and recall measured against an independent reading judge. None of the studies reviewed in Chapter 2 measures SOLID conformance on code that agents authored for real projects.

C2. A corpus of agent pull requests covering all three outcomes, which makes it possible to compare merged code with rejected and open code. Merged and unreviewed code are indistinguishable in Java and differ in Python.

C3. A description of which SOLID principles a structural detector measures reliably and which it can only screen, including two restrictions derived from labelled false positives, tested on a holdout and in a second language, and an explanation of why no width threshold can make the Open/Closed detector precise.

C4. Evidence, in two languages, that most design problems in the code agents touch were present before their changes, and that relation-level violations are rarely introduced by a single pull request.

C5. An observation about type systems and measurement: signals that the Java compiler rules out account for 46 of the 51 Liskov violations confirmed in Python, so the set of detectable violations depends on the language.

C6. An auditable validation procedure: every label must trace back to the source the judge was shown, every label has a recorded reason, and a mechanical audit enforces both.

C7. A study in two languages, Java and Python, with the same agents, which separates results that hold for agent-written code in both languages (the effect of change size, the Open/Closed difference, pre-existing violations, the behaviour of fitted rules) from results that do not (the outcome comparison, the precision of the size-based responsibility screen).

5.5 Practical Implications

For a team choosing a coding agent, the SOLID profile is an addition to functional benchmarks, not a replacement. The differences between agents are small and mostly reflect how much each agent changes at once. The one difference found in both languages is Claude Code's higher rate of Open/Closed flags. Across all three agents and both languages, classes the agents authored are flagged less often than the existing classes they edit, at about half the rate in Java and three quarters in Python on pooled rates, and at about a quarter and two fifths when averaged over repositories.

For a team building an automated review check, the results suggest one addition and one caution. The check worth adding is refused bequest: a subclass that overrides a working inherited method with a body that only refuses. It is cheap to compute, had precision 1.000 on 64 Java flags across two samples and six of seven in Python, and describes something a reviewer would object to. In a language without a compiler, a second check is that an override still accepts the calls its base accepts, since such faults made up most of the confirmed Python Liskov violations. An interface-width threshold does not transfer: it lost 16 points on the Java holdout and scored one of five in Python. A Single Responsibility or Open/Closed flag from detectors of this kind should prompt a discussion, not block a merge, because most such flags did not survive reading in either language.

Because the violations that matter most build up across changes, an automated structural check adds most where review of single changes cannot look: across the history of a component.

5.6 Limitations

SOLID adherence is not design quality. The thesis measures conformity to five principles; it does not measure whether conforming code is cheaper to change, less fault-prone or easier to understand. No step observes such an outcome. The link from a violation to later cost comes from the literature, which is not unanimous: Olbrich et al. (2010) find that God Classes are less change-prone and fault-prone than other classes once size is controlled. A reader who does not accept SOLID as a quality model can read the results as measurements of stylistic conformity; the comparisons between agents remain valid under that reading because the same instrument is applied to every agent. Linking these flags to later defects or changes in the same repositories would answer the question.

Every agreement figure is agreement with an LLM judge. Precision and recall measure agreement between a deterministic detector and a language model that reads the source. LLM judgement of SOLID violations has been evaluated by Pehlivan et al.

(2025), but the judge can still be wrong, including in the same direction as a detector. Two checks constrain this. For Java, 174 entities judged twice agreed on 89.7%; no Python entity was judged twice, so the Python judge's consistency is not measured. And for Java the class-level detectors agree with three independent analysers at least as closely as those analysers agree with each other. Interface Segregation has no counterpart in those tools, and the Open/Closed detector agrees with DesigniteJava's type-conditional smells at only $\kappa = 0.122$, so for these two principles, and for all Python figures, the judge is in practice the only check.

The judge and one agent come from the same model family. The judge is a Claude model and Claude Code is one of the agents. Per-agent precision gives no sign of leniency towards Claude Code (Table 4.9), but it cannot detect a bias shared by the judge and the detector.

Several agent coefficients are tested. The joint models test six agent coefficients in Java and ten in Python. With a Bonferroni correction, the Java Open/Closed coefficient ($p = 0.010$) would not pass alone and the Python import-cycle coefficient ($p = 0.0096$) does not pass. Only the Open/Closed difference, which passes in Python and has the same direction and a similar size in Java, is presented as a finding.

Change size is adjusted for, not explained. The agents write changes of very different sizes, and flag rates rise with size. The model adjusts for the number of files touched. If writing larger changes is part of how an agent works, the adjustment also removes that part of its effect. The model does not measure the kind of task each agent was given, which remains part of its coefficient.

Time is modelled as a linear trend. Median pull request dates differ by about seven months between agents in Java and about two months in Python. The time coefficient is close to 1 for every delta principle in both languages. A linear term cannot detect an abrupt change, such as a model release that altered one agent's behaviour, so the agent coefficients are adjusted for gradual drift only. The Python Single Responsibility trend rests on a flag with precision 0.110.

Rejected does not mean refused on quality. Most closed pull requests were never reviewed. Only the two reviewed strata, `changes_requested` and `reviewed_then_closed`, record a judgement, and they are small: 36 and 135 requests in Java, 33 and 163 in Python. The conclusions drawn from the outcome comparison are correspondingly narrow. In Java a merged-only corpus is not biased relative to unreviewed code; in Python merged code is cleaner than abandoned code. Neither shows what reviewers select on.

Reconstruction differs by outcome. In Java, 62.1% of merged requests are analysable against 56.8% of rejected ones, and the methods available to recover the before tree differ. The outcome comparison is therefore made only within matched methods.

Symbol resolution is incomplete. Median supertype resolution in Java is 87.0%, and 718 pull requests resolve below 50%, mostly in framework-heavy projects, where Liskov and Interface Segregation undercount. An unresolved supertype removes candidates and does not create false ones, so these rates are lower bounds. The Python extractor has no symbol solver and resolves a base class only when it is defined in the repository, so classes that derive from framework types contribute no override pairs or contracts; the error has the same direction.

One snapshot per pull request. The design compares two trees. Gradual erosion over many changes, which the locus-of-risk results suggest matters most for relation-level principles, is not observed.

Two languages from one dataset. Both corpora come from AIDev, so any property of how the dataset selects agent pull requests affects both. Two languages also do not establish generality: both are widely used, have large open-source ecosystems and use nominal class hierarchies, and a statically typed language with structural typing is not represented. A benchmark built on specific pull requests also decays: 18 Java requests could no longer be retrieved because repositories were deleted or their history rewritten.

Python loses source unevenly. 496 Python pull requests (7.0%) could not be analysed because their branch was deleted and neither a merge commit nor recorded commits remained. The loss is 11.0% for rejected requests against 5.1% for merged ones, and 14.3% for Copilot against 3.6% and 3.1% for the other agents. The lost requests were not replaced, so the Python outcome and Copilot figures describe the retrievable part of those populations.

The Python instrument is weaker in two places. The Single Responsibility screen is correct on 11 of 100 Python flags, so Python SRP rates describe large classes more than classes with several responsibilities. The Dependency Inversion validation confirms that each flagged pull request closes a new import cycle through changed code, but not that the cycle is harmful, and the most common form, a module importing its own package root, is accepted in many projects.

The two languages define two principles differently. Python has no `interface`, so its Interface Segregation flag requires width together with low implementer coverage or

a forced stub, while the Java flag can fire on width alone; and each language's thresholds were calibrated on its own corpus. Flag rates are therefore compared across languages only through the models and contrasts of Section 4.8, never as raw rates. The fall from 6.5% of Java pull requests to 0.6% of Python ones on Interface Segregation is largely a difference of definition.

A few Python classes are vendored. 489 of 124,143 Python class rows lie inside a virtual environment committed to a repository, 469 of them from one pull request, and 41 of the flagged SRP classes are among them. The production filter does not exclude committed environments; the effect on any pull-request-level rate is one pull request.

Small counts for relation-level principles. Sixteen Java pull requests introduce a refused-bequest violation and 195 a dependency cycle; in Python the counts are 47 for any Liskov signal (five of them refused bequest), 32 for Interface Segregation and 173 for dependency cycles. The Liskov and Interface Segregation agent coefficients have intervals several times wider than the Open/Closed one. Counts are reported next to every rate so that readers can judge them directly.

5.7 Future Work

- **A change-history signal for Open/Closed.** Dispatch chain length does not predict Open/Closed violations. A conditional that is edited in many pull requests to add cases is, by definition, not closed for modification, and a history of changes could measure this directly.
- **A closed-set suppressor for external specifications.** The current suppressor recognises only enumerations and removed none of the flagged validation rows. The closed sets that occur are JDBC type codes, protobuf scalars, Unicode blocks and generated parser tables in Java, and vendor API event types, protocol method names and grammar node kinds in Python.
- **An Open/Closed proxy that agrees with published smells.** The comparison with DesigniteJava reported in Section 4.7.2 reaches only $\kappa = 0.122$, and the detector fires three times as often. Requiring that a hierarchy already exist over the dispatched types, or that the chain sit in a class with no abstract supertype, would move the proxy towards the published definition and can be tested against the smells already recorded.
- **Recall for Liskov and Interface Segregation.** The weighted recall of these detectors depends on the few violations found in a large unflagged stratum. A

sample drawn from unflagged entities only would narrow the intervals; the three Liskov misses found so far suggest looking at silent refusals.

- **Judge consistency in Python and a second judge.** Re-judging a Python sample, and judging a sample with a model from a different family, would measure the judge's stability and its independence from the evaluated agents.
- **Why merged and rejected code differ in Python but not in Java.** A corpus with more reviewed rejections in both languages, and an analysis of what reviewers wrote when closing a request, would show whether the Python difference comes from review or from abandonment.
- **A framework-aware responsibility screen for Python.** Most Python SRP false positives are classes whose size a framework dictates. Recognising a class by the framework base it derives from could recover the precision lost between the languages.
- **A production filter for implementers.** Most Python Interface Segregation false positives are test doubles, null objects and unfinished scaffolds that stub a contract on purpose. Excluding implementers under test paths would remove the first group automatically.
- **A third language with a different type system.** A statically typed language with structural typing would test the compiler observation again, since such a language rejects some substitutability faults and not others.
- **Repository history as the unit of analysis.** Relation-level problems build up outside the reach of any single change. Measuring a component across its history is the natural next design.

5.8 Closing Remark

The central result of this thesis is a distinction. Most design problems that the detectors find in the code these agents touched were there before the agents arrived, and new violations appear mainly inside single units, where one change can create them. Relation-level problems accumulate across many changes and many authors. This held for three agents, for merged and unmerged work, in Java and again in Python, as did the finding that how much an agent changes matters more than which agent it is. The remaining question is what happens at a larger scale: a repository maintained over months by agents and by the people who edit their output.

Bibliography

Anthropic (2025). Effective context engineering for AI agents. Anthropic engineering blog, 29 September 2025.

Ben Sghaier, O., Li, H., Adams, B., and Hassan, A. E. (2026). Don't blame the large language model: How agent harness evolution shapes coding agent quality. *arXiv preprint arXiv:2607.03691*.

Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Chidamber, S. R. and Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493.

Cynthia, S. T., Muttakin, A., and Roy, B. (2026). Beyond bug fixes: An empirical investigation of post-merge code quality issues in agent-generated pull requests. *arXiv preprint arXiv:2601.20109*.

Fowler, M., Beck, K., Brant, J., Opdyke, W., and Roberts, D. (1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.

Henderson-Sellers, B. (1996). *Object-Oriented Metrics: Measures of Complexity*. Prentice Hall.

Hitz, M. and Montazeri, B. (1995). Measuring coupling and cohesion in object-oriented systems. In *Proceedings of the International Symposium on Applied Corporate Computing*, Monterrey, Mexico.

Huang, D., Zhang, J. M., Luck, M., Bu, Q., Qing, Y., and Cui, H. (2023). AgentCoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010*.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. (2024). LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*.

Kashif, S. M., Li, R., Liang, P., Tahir, A., Feng, Q., Li, Z., and Shahin, M. (2026). Beyond functional correctness: Design issues in AI IDE-generated large-scale projects. *arXiv preprint arXiv:2604.06373*.

Khomh, F., Di Penta, M., Guéhéneuc, Y.-G., and Antoniol, G. (2012). An exploratory study of the impact of antipatterns on class change- and fault-proneness. *Empirical Software Engineering*, 17:243–275.

Konrad, P. M., Adam, T. L., Terrenzi, R., and Ayvaz, S. (2026). Architecture without architects: How AI coding agents shape software architecture. *arXiv preprint arXiv:2604.04990*.

Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Prentice Hall, 3rd edition.

Le, T., Thai, M. V. T., Manh, D. N., Nhat, H. P., and Bui, N. D. Q. (2025). SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*.

Li, H., Zhang, H., and Hassan, A. E. (2025). The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003*.

Li, J., Deng, H., Zhang, Y., Zhang, K., Shao, T., Zhao, T., Wang, W., Jin, Z., Li, G., Liu, Y., Fang, Y., and Dong, Y. (2026). RealBench: A repo-level code generation benchmark aligned with real-world software development practices. *arXiv preprint arXiv:2604.22659*.

Lu, P., Zhang, S., Hou, Y., Ye, L., Huang, C., Chen, Z., Zeng, J., Jiang, H., Liu, P., Wang, Y., and Yang, M.-H. (2026). ProjDevBench: Benchmarking AI coding agents on end-to-end project development. *arXiv preprint arXiv:2602.01655*.

Marcus, A., Poshyvanyk, D., and Ferenc, R. (2008). Using the conceptual cohesion of classes for fault prediction in object-oriented systems. *IEEE Transactions on Software Engineering*, 34(2):287–300.

Marinescu, R. (2004). Detection strategies: Metrics-based rules for detecting design flaws. In *Proceedings of the 20th IEEE International Conference on Software Maintenance (ICSM)*, pages 350–359.

Martin, R. C. (2000). Design principles and design patterns. Object Mentor, white paper.

Martin, R. C. (2003). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.

Mazloomzadeh, I., Morovati, M. M., and Khomh, F. (2026). How do AI coding agents contribute to software development? an empirical study of agentic pull requests. *arXiv preprint arXiv:2607.21832*.

Olbrich, S. M., Cruzes, D. S., and Sjøberg, D. I. K. (2010). Are all code smells harmful? a study of god classes and brain classes in the evolution of three open source systems. In *Proceedings of the IEEE International Conference on Software Maintenance (ICSM)*.

Orlanski, G., Roy, D., Yun, A., Shin, C., Gu, A., Ge, A., Adila, D., Roberts, N., Sala, F., and Albarghouthi, A. (2026). SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. *arXiv preprint arXiv:2603.24755*.

Pehlivan, F., Ergüzen, A. U., Moslemi Yengejeh, S., Lami, M., and Koyuncu, A. (2025). Are we SOLID yet? an empirical study on prompting LLMs to detect design principle violations. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. arXiv:2509.03093.

Sawada, S., Shirai, T., Kashiwa, Y., Yamaguchi, K., Iwata, H., and Iida, H. (2026). To what extent does agent-generated code require maintenance? an empirical study. In *Proceedings of the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. arXiv:2605.06464.

Suryanarayana, G., Samarthyam, G., and Sharma, T. (2014). *Refactoring for Software Design Smells: Managing Technical Debt*. Morgan Kaufmann.

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160.

Whitfill, P., Wu, C., Becker, J., and Rush, N. (2026). Many SWE-bench-passing PRs would not be merged into main. METR research note, 10 March 2026.

Xu, F., Medappa, P. K., Tunc, M. M., Vroegindeweij, M., and Fransoo, J. C. (2025). AI-assisted programming decreases the productivity of experienced developers by increasing the technical debt and maintenance burden. *arXiv preprint arXiv:2510.10165*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Zhu, H., Zhang, Q., Wang, J., Yang, Z., and Qiu, Y. (2026a). Needle in the repo: A benchmark for maintainability in AI-generated repository edits. *arXiv preprint arXiv:2603.27745*.

Zhu, Y., Tsantalis, N., and Rigby, P. C. (2026b). AI-generated smells: An analysis of code and architecture in LLM and agent-driven development. *arXiv preprint arXiv:2605.02741*.

Zhuo, T. Y. et al. (2025). BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations (ICLR)*.

Chapter

A

Pipeline Run Order and Reproducibility

This appendix lists the order in which the stages were run to produce the results in Chapter 4. Every stage can be resumed. After a threshold change, the stage's output Parquet file must be deleted before rerunning, because resumption otherwise keeps rows computed with the old threshold.

```
curate.py --lang java      -> gold set        6,956 PRs / 1,010 repos,
                                              per agent: all rejected and
                                              open, merged at 2x rejected
clone_and_extract.py       -> file lists      GitHub API where local
                                              history is insufficient
mvn -o package (sidecar)   -> facts jar       JavaParser + SymbolSolver
run_java.py                -> five parquets   one parsed tree feeds all
                                              five principles; 4,245 ok
   (thresholds calibrated from corpus percentiles, then parquets
    deleted and the pass re-run, so no flag predates its threshold)
report_java.py             -> per-agent and per-outcome summaries
fit_mixed_model.py         -> the joint model
make_java_labeling_sample.py  -> 100 flagged + 100 unflagged per principle
extract_java_label_batches.py -> the source shown to the judge
audit_label_batches.py     -> GATE: every row must trace to that source,
                              and no batch may show the detector's verdict
score_labels.py            -> precision and recall per principle,
                              per agent, per outcome
run_crosscheck_java.py     -> CK, PMD, DesigniteJava over the same trees
verify_v2_java.py          -> whole-corpus invariants; exits non-zero
                              on any failure
```

One combined pass (`run_java.py`) computes all five principles from a single parse of each tree. Extracted facts are cached by the content hash of the tree, so a rerun cannot silently use a different input.

The Python study (Section 4.8) follows the same order with its own extractor and calibration. Its label files carry a `py2_` prefix, and every labelling tool is called with `--lang py2`, so a Python label cannot be written into the Java study.

```
curate.py --lang py        -> gold set        7,050 PRs / 3,974 repos,
                                              per agent: 1,400 merged,
                                              700 rejected, 250 open,
                                              stratified by repository
clone_and_extract.py       -> file lists      same recovery as Java
run_python.py --calibrate  -> all metrics, flags switched off
propose_py_thresholds.py   -> thresholds near the 90th percentile,
                              written to config and re-imported
   (calibration parquets deleted before the flagged pass)
run_python.py --shard i/6  -> five parquets   repository-disjoint shards,
merge_py_shards.py            merged under refusal checks; 4,946 ok
report_python.py           -> per-agent and tier-matched outcome summaries
fit_mixed_model.py --lang py  -> the joint model, all five principles
make_py2_labeling_sample.py   -> 100 flagged + 100 unflagged per principle
                                 (ISP: all 80 flagged, a census)
extract_py2_label_batches.py  -> the source shown to the judge
audit_label_batches.py --lang py2  -> the same GATE as Java
score_py2_labels.py        -> precision, stratum-weighted recall,
                              per agent, per outcome
py2_cross_language.py      -> the Java results tested in Python
verify_thesis_numbers.py   -> every headline number in the thesis
                              recomputed from the data
```

The Label Judge

Every other component of the pipeline is deterministic: the same tree gives the same facts and the same facts give the same flags. The label judge is the exception, so its configuration is recorded here.

What the judge is. A large language model of the Claude family that reads the source of one entity at a time and returns a binary label with a written reason. Java judging took place between 5 and 8 September 2026 and Python judging on 14 September 2026, each under a written rubric fixed before judging began. Each borderline Python decision was recorded as a numbered precedent and applied to later rows. The judge computes no reported number; its labels enter the study only as the reference for Section 4.7.1.

What it was shown. For each sampled entity, the extractor writes a batch file with the entity's source and, for relational principles, the source of the other side: the base method next to the override for Liskov, the contract next to each implementer for Interface Segregation. The detector's verdict is removed from this material, and the audit refuses to start judging if any batch still contains it.

What a label must contain. Labels are written only through `record_labels.py`, which refuses a row whose source file is missing, a label that is not boolean, and a label without a reason. All 1,495 Java and 980 Python labels therefore carry a reason recorded at judging time, and the reasons are stored with the labels in the sample files.

A consistency check. The Python samples were shuffled before judging, so the share of positive labels in the first and second half of each stratum should be similar. Four principles meet this. For Interface Segregation the flagged stratum reads 0.513 in the first half against 0.341 in the second. Every flagged row was read again against the recorded precedents and no rule had been applied differently; the gap comes from three contracts that each occur twice, from forked or related repositories, all in the first half. No label was changed.

What cannot be reproduced exactly. A language model is not a deterministic function of its input, so judging again would not reproduce every label. Two things limit the effect. The sampling is seeded, so the same rows are drawn every time. And for Java, 174 entities judged twice agree on 89.7%, which bounds the accuracy of any agreement figure. That comparison uses the first labelling round, which was discarded as a source of results because its provenance could not be audited: 294 of its Interface Segregation labels belonged to rows whose batch file did not exist, and 200 more were written at about three seconds each. Those labels measure the judge's stability here and contribute to no reported rate; the guards described above exist because of that round. The comparison with CK, PMD and DesigniteJava provides a check that does not depend on the judge at all.

Conflict of interest. The judge belongs to the same model family as one of the three agents. Section 4.7.1 tests for leniency towards that agent; the Python per-agent precision is in Table 4.25.

Software and Data Availability

The corpus is derived from the public AIDev dataset (Li et al., 2025). The pipeline, its configuration, the result tables and the labels are under version control. No proprietary code, personal data or human-participant data was collected or is distributed.

Configuration

All thresholds, file paths and policy settings (agent sets, star and licence filters, rubric thresholds, suppressor switches) are defined in one commented configuration module, so

a threshold change is a single edit. Each pure detector module has a self-check, a small example with a known answer that runs when the module is executed as a script; the pipeline uses these instead of a separate test framework.